

Natural Language Processing in Finance — Unit 4

Table of Contents

- Tool: Unit 4 Glossary

Module Introduction: Train and Evaluate a Doc2Vec Model

- Watch: Introduction to the Doc2Vec Algorithm
- Watch: Set Up the Data for Doc2Vec
- Watch: Doc2Vec Parameters and Vector Output
- Code: Running Your First Doc2Vec Algorithm
- Watch: Introduction to Dimension Reduction
- Watch: Visual Interpretability — PCA
- Code: Implementing PCA
- Watch: Visual Interpretability — TSNE
- Code: Implementing TSNE
- Module 4.1 Assignment: Training and Evaluating a Doc2Vec Model
- Module Wrap-up: Train and Evaluate a Doc2Vec Model

Module Introduction: Use Doc2Vec Loadings to Find Comparable Companies

- Watch: Use Distance Metrics to Find Comparable Companies Part 1
- Watch: Use Distance Metrics to Find Comparable Companies Part 2
- Code: Using Distance Metrics to Find Comparable Companies
- Discuss: Doc2Vec vs. GICS vs. LDA
- Watch: Use Doc2Vec to Find Similar Words through Semantic Vectors
- Code: Finding Comparable Words



- Watch: Use Doc2Vec to Find Companies Aligned with Semantic Vectors — BOTZ
- Watch: Using Doc2Vec to Find Companies Aligned With Semantic Vectors — ARKG
- Code: Using Doc2Vec to Find Companies Aligned With Semantic Vectors
- Discuss: Thematic Portfolio Construction
- Module 4.2 Assignment: Using Doc2Vec Loadings to Find Comparable Companies
- Module Wrap-up: Use Doc2Vec Loadings to Find Comparable Companies

Module Introduction: Apply the Doc2Vec Algorithm

- Watch: Introduction to Clustering Algorithms
- Watch: Evaluating Clustering Algorithms
- Code: Evaluating Clustering Algorithms
- Watch: Putting it All Together
- Code: Ensembling for Improved Clustering
- Module 4.3 Assignment: Applying the Doc2Vec Algorithm
- Module Wrap-up: Apply the Doc2Vec Algorithm



Faculty Author



Mr. Chris Meredith

Senior Visiting Lecturer

SC Johnson Graduate School of Management

Cornell University

Mr. Chris Meredith is a senior portfolio manager and the Director of Research and Portfolio management at O'Shaughnessy Asset Management (OSAM). Mr. Meredith reports to the CEO/CIO, and is responsible for managing the investment activities of the firm. This includes supervising the portfolio management team, investment strategy research, and overseeing the firm's trading efforts. His portfolio management responsibilities include daily model generation, strategy optimization, reviewing account rebalances, and trade analysis. On the research side, Mr. Meredith leads a team of analysts that conducts research on new and existing strategies and evaluates the efficacy of new factors. Prior to joining OSAM, Mr. Meredith was a senior research analyst on the Systematic Equity Team at BSAM. He was a director at Oracle Corporation and spent eight years as a technology professional before attending the Cornell SC Johnson College of Business.

Mr. Meredith holds a B.A. in English from Colgate University, an MBA from Cornell University, an M.A. in financial mathematics from Columbia University, and is a Chartered Financial Analyst. He lives in Chappaqua, New York, with his wife and three children.



Unit 4: Doc2Vec Functional Overview

Video Transcript

One of the key decision points in an ML project is the algorithm selection. Up until now, we've been using the LDA as the only algorithm to determine topics. But we've been doing that with the idea of building other skills such as preprocessing the data, working with pandas, preprocessing the text, developing with regex, going through with lemmatization, stemming, and getting to the core concepts and then having it where we're doing hyperparameter tuning and model evaluation. Now we're going to introduce a new algorithm and evaluate its utility in determining topics in competitive companies. With that, we're going to introduce a few other concepts as well within this set of lectures such as dimension reduction, data visualization, clustering algorithms. All of these can be applied to areas beyond just topic modeling and will expand your data science skills.

What you'll do

- **Train and evaluate a Doc2Vec model**
- **Use Doc2Vec loadings to find comparable companies**
- **Apply the Doc2Vec algorithm**



Unit Description

The LDA algorithm can be a powerful tool for text data analysis, but it has certain weaknesses that bear consideration before applying results in a real-world context. For that reason, it can be helpful to explore additional algorithms to compare and contrast performance and application before settling on one method for any natural language processing project. Doc2Vec is another algorithm often used for text data analysis that offers a unique approach, creating numerical vectors that capture the context and relation of words to documents, rather than generating topics by word



frequency. While this approach has its own weaknesses, it also offers certain strengths that are highly relevant to the construction and management of investment portfolios.

In this unit, you will explore the Doc2Vec algorithm as an alternative approach to text data analysis, replicating many of the same general operations that you performed with the LDA algorithm. You will go from training and evaluating an initial Doc2Vec model, to constructing your own custom vectors to build lists of comparable companies relevant to specific investment themes. Along the way, you will introduce additional algorithms as a component of that analysis, exploring different ways to customize and visualize results for comparison against an industry standard and real-world investment portfolios.

[Back to Table of Contents](#)



Tool: Unit 4 Glossary

Most of the terms presented throughout the unit are defined and described in context, but there are a few vital terms that require further elaboration. While you won't need to know these terms until later in the unit, it can be helpful to glance at them briefly now using the link to the right. Use the unit glossary to help you recognize important terms and facilitate your understanding.



Download the Tool

Save this **Unit 4 Glossary** tool to use as a reference as you work through the unit and encounter unfamiliar terms.

[Back to Table of Contents](#)



Module Introduction: **Train and Evaluate a Doc2Vec Model**



While the LDA algorithm can be useful for a variety of business applications, overall efficacy can be limited by its reliance on word frequencies. The Doc2Vec algorithm, on the other hand, does not rely on word frequencies at all. It instead converts words and documents into numerical vectors that capture the context of words and their relationship to documents. That difference allows for a number of unique use cases that would have been difficult, if not impossible, to achieve with the LDA algorithm alone.

In this module, you will prepare data for the Doc2Vec algorithm and train your first model, converting business descriptions into numerical, semantic vectors for analysis. You will then mathematically reduce the dimensions of Doc2Vec output through two different methods, comparing and contrasting their semantic relevance to company and sector assignments. Finally, in the module assignment, you will train and evaluate a Doc2Vec model for a business description corpus, incorporating dimension reduction to produce two different data visualizations for evaluation of model performance and alignment with GICS.

[Back to Table of Contents](#)



Watch: Introduction to the Doc2Vec Algorithm

Unlike the LDA algorithm, which groups similar words into topics by frequency for assignment to documents, the Doc2Vec algorithm vectorizes word space, translating documents into numerical vectors. This provides unique benefits, as the context of words and their relation to documents is captured by resulting vectors. In this video, Mr. Meredith emphasizes this distinction, exploring potential use cases best suited to each algorithm.

Video Transcript

As a data scientist, one of the most important decisions that's made in any machine learning project is the selection of the algorithm that's being used. What you want to have is an alignment between the algorithm and the function that it performs and the activity you are asking it to do, which is what you're trying to have it predict.

We want to introduce in this set of lectures the doc2vec algorithm. Doc2vec is different than the LDA algorithm that's been presented before, in the sense that doc2vec actually vectorizes word space, translating a document into a numerical vector of loadings. Think of this like a linear regression, where normally you would have an input of a set of data to try to bring out what might be some explanatory variables inside of it. But instead, this algorithm is creating semantic vectors. And think of semantic in that sense where the words have a meaning attached to them. It's translating it into a numerical landscape, where it's going to have exposure to what those senses or those semantic meanings are.

By looking at the probability distribution, the words within the larger corpus, you can have it where you are taking those words and those meanings out of it, and having it where it will give you exposures of a certain amount, like a 2, 4, 5, numerically of what it is to the meanings of the words themselves. The LDA algorithm was looking at word frequencies, and it was taking in building the distributions of the words and comparing it to the distributions of the topics, and generating those topics that came out of it. This is going to be organizing the information in a different way, which can lead to different utilities and benefits that come out of the algorithm.



The idea of having the LDA algorithm giving you the frequency and the topics potentially has lent itself to a strength of things like topic creation. Whereas the doc2vec, because of the numerical landscape where it's continuous numbers, can have other benefits potentially on what we're going to be doing with, say, competitive companies. What we want to do is take this where we're going to walk through the benefits in it — first set up the doc2vec algorithm and then talk through some of the benefits within those various areas or some of the challenges that it may have.

[**Back to Table of Contents**](#)



Watch: Set Up the Data for Doc2Vec

Though similar to the work you did for the LDA algorithm, preparing data for the Doc2Vec algorithm requires a few additional steps for business descriptions to be effectively converted into numerical vectors. In this video, Mr. Meredith outlines that process, highlighting areas of similarity and difference with LDA to numerically encode text data as input to the algorithm.

Video Transcript

We're going to leverage the work that was done in the previous sets of lessons to set up the data and get it towards the doc2vec algorithm so we can train that algorithm utilizing the business descriptions that were cleaned before.

For this, it's fairly mechanical. We're going to go through, take our code, we'll have our — first inside of this new Jupyter Notebook — we're going to have it where we import all the packages that are relevant that we need to utilize.

We're then going to go back to our `import pickle`, where we're going to load up the data that was utilized in the previous set of lessons. We're then going to do a few things on cleaning this where we're going to make sure there's a GICS industry attached to it, a market cap greater than zero, making sure that we have more than five words inside of a business description, and making sure we're adding back in that industry group description where, again, it's the GICS industry at a six-digit number, dividing it by 100 to get it to the four-digit industry group.

With that, we'll have our dataframe where you can see that we've got our `companyid`, `gvkey`, `compustat_name` — `gvkey` being the compustat identifier. We're going to include the `ticker`, `mktcap`, `gics` — the sector all the way down to the subindustry — the description, which is the raw original one which we want to preserve so as we go through and analyze the companies we can read it more cleanly, then the clean description, which is where it's all of the stripped-down words, where we've taken out stopwords punctuation, and all the rest of it.

We keep the clean sentences and the `nameWords` and `nWords` where the name words were the ones that we stripped out the names from them. Not necessarily useful, but



the sentences are, as we're going to continue to apply that, inside of `doc2vec`. The reason for that, as mentioned, `doc2vec` does take the word order into account for each of the sentences. We want to make sure that we have these sentences that are put in there. We're going to create a new field called the sentence list. That will get utilized we'll see when we get to feeding this into the Doc2Vec algorithm.

We're also going to go back and include in those industry group names where we have, instead of just a four-digit code here, we're going to actually have `Energy`, `Materials`, `Capital Goods`, and use that. So it will give us the sense for what's inside of each of these industry groups.

We're then going to sort it by market cap and reset the index. That way it's going to be joined where top to bottom the vectors that we're going to use are going to align with what's inside of the dataframe. I will say having it where the index and the vector being misaligned has caused problems for me in the past. So this is the part where you're going to order it and do the `longD`, reset the index, dropping the index so that way these numbers are going to be ordered in the way that they are inside of the dataframe as well.

With that, then the next step, really this is important for Doc2Vec in particular, which is setting up the docs that are going to be fed into this. This is where there is a special within a `gensim.model.doc2vec`, this `TaggedDocument`. What's going to happen is we're going to create an object for it that is `docs`. It's going to be the tagged document collapse nested for the clean sentence list with the location. We're going to do that for everything that's inside of the `longD`. For every one of those, we're going to create this set of `docs`. That `docs` is going to be the key input that we're going to use inside of the Doc2Vec algorithm.

[Back to Table of Contents](#)



Watch: Doc2Vec Parameters and Vector Output

Earlier in this course, you spent time iteratively adjusting and testing hyperparameters for the LDA algorithm to improve model performance. In this video, Mr. Meredith introduces that same process for Doc2Vec, explaining how each model setting affects vector output. While there is equivalent value to an iterative adjustment/testing process for the Doc2Vec algorithm, default settings will be provided to you for the purposes of this course.

Video Transcript

So now we have our data set up for the Doc2Vec algorithm. We went back and we took the clean sentence list, which you can see here, which is a list of lists of words: `pple, incorporated, designs, manufacturers, markets`. We put those into the `docs`, which is a set of `TaggedDocuments`, which is a gensim.model object for the TaggedDocument, which is then utilized to feed into the Doc2Vec algorithm.

Now, the Doc2Vec algorithm, setting it up and training it is incredibly simple. We have here this one `d2v = Doc2Vec` object that is from the gensim object, having it where we set some parameters. We have the `d2v.workers`, which is simply the number of threads that we're going to be doing. We build the vocabulary from the `docs` and then we train it, and after that it's completed. The important thing is knowing what these hyperparameters are.

If you remember, the exercise of hyperparameter tuning, you may want to go back and iterate and pull some of these levers and see what the outcome and the effect is. In this case, by the way, if you would like to understand all the levers you can go online and see there's typically documentation out there for the `gensim.models.doc2vec.Doc2Vec` object and all of the pieces of what you can go through and adjust inside of it. Again, utilize the internet to understand these things.

For us, the most important one is this `vector_size`. I mentioned it's like a linear regression where you're going to get out a number of variables. In this case we're going to set 150. Think of it as 150 different semantic loadings, or meanings, that we're



trying to pull out of this set of documents. In each one of those, it'll get a numerical representation for what's inside of those vectors.

We're going to then have a `min_count` where we're going to ignore all words with a total frequency lower than that; the `epochs`, which is like the iterations; a `window`, which is looking at the probability distribution between words, it's a maximum distance between the current predicted words within a sentence, so no more than eight words that we're going to be looking; and a `sample`, which is the threshold for configuring higher frequency words which are randomly downsampled. Again, this is one where we're just keeping this as a trim number, `1e_5`, so it's an exponent down. We're going to run this, and it's fairly quick.

What we get out of this when it's completed is going to wind up looking like this where we're going to get this vector: for `f in longD`, the clean description. We're going to take that `vec` and append it the `d2v.docvecs`. What you'll see is the very first — I'm going to count first of all the length of this, which is we've got a vector for every single one of our companies that's inside of our dataframe. I just print the first one to show you what it looks like, which is this first vector where we get the numbers.

In this case, you can see it's fairly straightforward. 0.36, -0.44, -1.0. Again, there's no interpretability. You can think of it as blind factors where we've got one to 150, but we've got that array for, in this case it's `Apple` that's our very first one. What we're going to do is then take this and feed it back into our dataframe.

Down here we're going to work through this code and make sure that we preserve it and we have that `vDf`, which is a data frame for doc2vec. I'm making a copy of the data frame simply to make sure that we can play around with this thing. This happens a lot in Jupyter where you do something and you want to undo it, so you keeping a copy of the data can be somewhat helpful in this case. We're going to have our `logmcap`, which wasn't included the last one. I'm just adding that back in.

This is one where for the range in the `len(d2v.docvecs)`, we're actually going to create a series of fields in our data frame. This is a quick way to go through and add to the data frame 150 columns where for each one of these we're going to go a `d2vDf`. We're going to add the column `d2v` with the string for the range. In this case it's going to be zero to the length of the doc2vecs. We're then for everything that's after the 17th column add `vec`. That's going to come through and append it to all of those columns that are



coming through. What you can see here is for `Apple` we have the `company_id`, `gvkey`, etc. It's truncating the data on the print, but you can see clearly we have a `d2v140`, `141`. We're getting those numerical representations that are coming through from our `doc2vec`, which we can now start to unpack and utilize in our analysis.

[Back to Table of Contents](#)



Code: Running Your First Doc2Vec Algorithm

This is an ungraded practice activity.

While the LDA algorithm can be effective in analyzing business descriptions and generating topic associations by activity on that basis, there are a variety of other models available for textual analysis.

As you have seen, the Doc2Vec algorithm vectorizes word space, translating documents into a vector of loadings. In doing so, the algorithm preserves the order of words to generate semantic vectors, drawing associations between companies by topic similarity rather than word frequency as the LDA algorithm did. A major benefit of this continuous numerical landscape is the ability to mathematically reduce similar words or documents, allowing for numerous additional operations to be performed.

In this activity, you will train your first Doc2Vec model on the business description corpus of the Top 1000 companies, verifying that the resulting vector of loadings is merged back into the dataframe correctly for analysis in future activities.

Activity Instructions: Using the sandbox below, address the following tasks and questions to train your first Doc2Vec model on the cleaned business descriptions from Unit 2, merging the loadings from that model into those descriptions to create vectors specific to each of the Top 1000 companies. Once you feel confident in your response, click the **Solution** button located beneath the sandbox to reveal the answers.

1. Import the `cleanDesc` business descriptions for the Top 1000 companies from Unit 2.
2. Split the description into sentences for utilization by the Doc2Vec algorithm.
3. Merge in the GICS industry group names and print the business descriptions, sorted by market capitalization.
 - List the top 10 companies and their associated GICS industry groups.
 - Do you agree with those GICS classifications?
4. Train your first `d2v` model, merge the resulting loadings back into the `longD` dataframe, and print the loadings for the first vector.
 - List the first 6 loadings for the first vector.



- How many loadings are in the first vector?
5. Print the `d2v` loadings for Apple to verify that your merge of loadings into the dataframe was effective.
- List the first 6 loadings for the `AAPL` vector.
 - Do those loadings match the loadings for the first vector?

Note: Output will be different for each run of the model; hence, your results may not reflect the activity solution.

Please access this Jupyter Notebook exercise in the course.

Solution



3. Merge in the GICS industry group names and print the business descriptions, sorted by market capitalization.

a. List the top 10 companies and their associated GICS industry groups.

Answer: Apple (Technology Hardware & Equipment); Microsoft (Software & Services); Alphabet (Media & Entertainment); Amazon.com (Retailing); Tesla (Automobiles & Components); Meta Platforms (Media & Entertainment); NVidia (Semiconductors & Semiconductor Equipment); Berkshire Hathaway (Diversified Financials); UnitedHealth Group (Health Care Equipment & Services); JPMorgan Chase (Banks).

b. Do you agree with the GICS classifications?

Answer: Yes. While there may be an opportunity for more granular distinctions, the GICS classifications for these well-known companies broadly make sense.

4. Train your first `d2v` model, merge the resulting loadings back into the `longD` dataframe, and print the loadings for the first vector.

a. List the first 6 loadings for the first vector.

Answer: `0.0168004, 0.431058, -0.619611, -0.257848, -0.432121, 0.764539`.

b. How many loadings are in the first vector?

Answer: 150, which makes sense given settings used to train the model.

5. Print the `d2v` loadings for Apple to verify that your merge of loadings into the dataframe was effective.

a. List the first 6 loadings for the `AAPL` vector.

Answer: `0.168004, 0.431058, -0.619611, -0.257848, -0.432121, 0.764539`.

b. Do those loadings match the loadings for the first vector?

Answer: Yes, which means that vectors were correctly inserted back into the dataframe. Checking that data manipulations and transformations are valid is always good practice, as in this case results would have turned into noise if the index wasn't aligned properly with the order of the dataframe.

[Back to Table of Contents](#)



Watch: Introduction to Dimension Reduction

Now that your Doc2Vec algorithm has been trained, you might have noticed the complex, high dimensional nature of resulting vectors. Each of those dimensions represent a semantic feature of a word (sentiment) or document (topic) that capture granular nuances in the data, but make it incredibly difficult to interpret. In this video, Mr. Meredith introduces principal component analysis (PCA) as a method for reducing that dimensionality, simplifying output for enhanced interpretability.

Video Transcript

Before we start utilizing the Doc2Vec algorithm and the output from it, we need to evaluate how well is it doing on mapping, what are those semantic senses to potentially what's going on inside of the meaning of this?

Numerical representations, we talked about some of the gaps in that and how it can be better with using human judgment to go through and analyze the algorithm and how well it's doing. But now we're dealing with Doc2Vec where before we were dealing with word frequencies and we could get some sense of the words are aligning with the topics that we would expect. Now we're dealing with mathematical loadings that are in 150 dimensions. We have to figure out how to try to make sense of this and compare it against something that is interpretable.

We need to build a framework to determine whether this model is being effective. What we want to do is try to compare what's coming out of the Doc2Vec against GICS and see if there's any alignment between those GICS classifications, which are done by committee, human people that are out there and saying these are companies that have this type of a business activity. We're now taking our business descriptions and we're putting them in through the Doc2Vec algorithm and coming out again with 150 dimensions that we want to create the numerical loadings and evaluate.

Now, that challenge on 150 dimensions — it's incredibly difficult for the human mind to process. So what we're going to introduce is the concept of dimension reduction. We can't process 150, but we can process two or three — three in this case visually. What we want to do is figure out if there's an algorithm we can use, and we're going



to use two different ones to do dimension reduction to take the 150 dimensions and reduce it down to three.

We're first going to introduce principal component analysis. It's one of the strongest techniques for dimension reduction because first in any dimension reduction — you're going from 150 numbers down to three — you're going to lose information. You're going to lose something that is important inside there. But PCA, principal component analysis, is actually one of the strongest algorithms because it minimizes that information loss. It uses linear methods. It goes through and maximizes the variance inside of the reduced dimensions, and through that it preserves a great deal of information. In that sense, I would highly recommend if you're more interested in the algorithm itself and how it works mathematically, go out to sites like Wikipedia where there is a high degree of people that have contributed to the literature describing this and how it gets utilized. But with that, we're going to take this and now show you how to utilize PCA to get to the interpretability of our algorithm.

[**Back to Table of Contents**](#)



Watch: Visual Interpretability — PCA

Once Doc2Vec loadings have been reduced from 150 dimensions to 3 with the PCA algorithm, not only is interpretability vastly improved, but vectors are ideally positioned for data visualization as well. In this video, Mr. Meredith explains how that algorithm can be used for dimension reduction, visualizing resulting vectors using Plotly in three dimensions to evaluate alignment of Doc2Vec output with GICS sectors.

Video Transcript

Twenty years ago, doing a PCA analysis could've been incredibly difficult for you. You'd have to go through and mathematically compute what the eigenvectors are within the covariant space of the 150 dimensions. But now, the beauty of open-source software is we can get to this in a fairly straightforward approach where you can use this in a practical sense. Again, I'm not going to mathematically be somebody that's going to create the PCA algorithm, but I'm going to show you how to utilize it, and it's actually an incredibly small amount of code.

What we're going to do is from the `sklearn.decomposition` package we're going to `import PCA`, which stands for the principal component analysis. We're literally going to just say, give me a PCA algorithm that's got three components. We're going to do inside of that a `fit_transform` on this vec, which is the same vector that we utilized before. Mathematically, you remember up here, we went through and said, give me all the doc2vec docvecs and put it inside of this array. The first one was these 150, but we have 3,738 rows each with those 150 dimensions.

What that's going to get reduced down to inside of this is this PCA result. The PCA result, when we print it you can see it's still going to have that same number of rows, the 3,700+, but instead of it, you're going to have just three numbers, in this case, -8.69, 1.24, 1.84. We're going to attach those back to our dataframe, the `d2vdf` with our `PCA xyz`. We're going to take the PCA result and do the first, second, and third columns of that vector. Then we're going to put that back in the dataframe and preserve that.



For this, then, we're going to just check on it and we can say, OK, how are we looking for Apple? You can now see we have xyz coordinates for x being -0.86, y being 1.24, z being 0.184, which aligns up here with that first row, so we feel like we're checking and preserving that the data order is accurate.

With that now, we want to take these three dimensions and put it towards a visualization, and we're going to use something called `plotly`, where we're going to have here `plotly`, `plotly.express`. We're going to create from that a `px.scatter_3d`. That scatterplot's going to use the dataframe, the `d2vdf`. It's going to have for the x, y, and z what your columns are, `pcax`, `pcay`, `pcaz`. For this, in the case of Apple, the x-coordinate will be that -0.86, the y, 1.24, the z, 0.184.

We're going to attach a color scheme to this and this idea of that visualization, so the xyz will represent what's coming out of solely `doc2vec` and the interpretation from it. The color will come through as our GICS industries, which if you remember, those four-digit codes from 1010 all the way up to 6010. We're going to keep in a fun thing here with the hover-over name, where you're going to get the `compustat_name`. We're going to do the size on that log of market cap so that bigger companies on market capital will be a little bit bigger on our visualization. We're going to put a cap on how big they're going to get. We're going to give you a height and width for what the output would be, as well as an opacity, making sure that it's not transparent. We're going to do the figure one, where it's going to update the traces. In this case, we're going to put a line around it that's dark slate gray to define some space around each of the visualizations, the spheres that we're going to do. Then we're going to have that figure and write it out. We're going to get a `div_pca.html` and show it.

We can also see it inside of the Jupyter Notebook itself right now. You can see this three-dimensional representation, the color scheme on the right. This dark bluish — I call it that way — is our energy sector, and when we move up to a little lighter purple shade, it winds up being our materials. This is me knowing that numerical representation of what we're working with. The 20 being the industrials, 25, the lighter purple, being consumer discretionary to the consumer staples when we get to that violet range. Health care is going to be a wine color in that 35, that range, 3,500. 40 is going to be our financials. We're going to move up a little further. Our technology is going to be more of that orangeish color. We're going to have our communication telecom services in that yellowish, or should I say burnt orange, I don't know what we



call it, maybe maize? My colors are off. And we've got the yellow here, which is going to be our real estate.

When we start looking at this, we can turn it around and start doing analysis of how this is looking inside of our PCA. You can see there's actually some really strong groupings coming through on this. Down at the bottom you can see a little bit of what's going on with your utilities in this with Entergy, CenterPoint, those are clustered at the bottom. The utilities are pretty close to energy, which makes sense. In this case we can see the Eagle Materials, that's a materials company in there.

We can zoom in on this, take a look and see over here there was Freeport-McMoRan, Mosaic. We've got a cluster over here on the energy side. Over here you can see on this are technology companies with a 45 that's got a strong grouping. Health care, a very strong grouping. Over here you can tell there's a carveout throughout this thing. Your health care companies over here with your Ionis Pharmaceuticals.

But you're seeing strong groupings between what you're getting out of the colors versus the geometric three-dimensional space that's coming through in this. This is telling me visually through the human judgment, we've got a pretty good alignment with our Doc2Vec algorithm, even after the dimension reduction of taking this down and in alignment with what's going on with the GICS and the human committee-based classification of the business descriptions.

[Back to Table of Contents](#)



Code: Implementing PCA

This is an ungraded practice activity.

In order for you to determine if your Doc2Vec model is effective, you need a framework for analysis. However, due to the numerical nature of Doc2Vec loadings, interpretability of semantic vectors can be a significant challenge. In order for you to effectively make comparisons against GICS or any other classification method, you must find a way to make those loadings more interpretable. Fortunately, the numerical landscape created by the Doc2Vec algorithm allows for mathematical reduction of similar words or documents.

In this activity, you will use the PCA algorithm to reduce the dimensions of Doc2Vec loadings from 150 to 3, which will enable you to use Plotly to visualize semantic vectors in three dimensions. Once this is done, you will be able to assess alignment of Doc2Vec with GICS, similar to evaluations you conducted for LDA models in previous lessons.

Activity Instructions: Using the sandbox below, address the following tasks and questions to reduce the loadings of the Doc2Vec model from 150 dimensions (loadings) to 3 with PCA, then use Plotly to visualize the resulting vectors of the Top 1000 companies. Once you feel confident in your response, click the **Solution** button located beneath the sandbox to reveal the answers.

1. Run `sklearn.decomposition.PCA` on `d2v` vectors to reduce their dimensions (loadings) from 150 dimensions to 3.
2. Audit those loadings to verify reduction to 3 dimensions.
 - List PCA vector loadings of the top 10 companies by market cap.
3. Run Plotly to build a 3D scatter plot of the `d2v` vectors.
 - Can you see some separation between GICS groups in the chart?
 - Is there some crowding around the center?
 - How effectively do PCA reduced Doc2Vec loadings capture the underlying meaning of business descriptions?



Solution

2. Audit those loadings to verify reduction to 3 dimensions.

a. List PCA vector loadings of the top 10 companies by market cap.

Answer: Apple $x: -1.008, y: 1.120, z: -2.830$; Microsoft $x: -5.404, y: 5.609, z: -4.834$; Alphabet $x: -1.659, y: 3.340, z: -3.033$; Amazon.com $x: -2.23, y: 0.901, z: -2.714$; Tesla $x: -3.138, y: -0.936, z: -3.128$; Meta Platforms $x: -1.351, y: 3.829, z: -2.530$; NVidia $x: 0.351, y: 7.517, z: -10.631$; Berkshire Hathaway $x: -2.817, y: -8.54, z: 4.33$; UnitedHealth Group $x: 3.021, y: 6.342, z: 8.072$; JPMorgan Chase $x: -3.369, y: -0.519, z: 4.016$.

3. Run Plotly to build a 3D scatter plot of the **d2v** vectors.

a. Can you see some separation between GICS groups in the chart?

Answer: Yes, when rotating the chart you can see a clear separation by GICS color schemes. Health care companies have the most separation, but groupings for technology, energy, and utilities are also fairly clear.

b. Is there some crowding around the center?

Answer: Yes, there is some crowding around the center, implying that dimension reduction via PCA was not entirely effective from a visual perspective in separating companies by industry or activity type.

c. How effectively do PCA reduced Doc2Vec loadings capture the underlying meaning of business descriptions?

Answer: While there is already an acknowledged loss of information as a result of dimension reduction, it is clear from the visualization that the Doc2Vec algorithm is providing numerical representations of companies' descriptions sufficient to separate them by industry or activity to an extent (e.g., health care, energy, technology). One could even infer that Doc2Vec is acknowledging some similarity between Energy, Materials, and Utilities by their closeness, in contrast with Health Care which is by far the most distinct group by activity.

[Back to Table of Contents](#)



Watch: Visual Interpretability — TSNE

The T-distributed Stochastic Neighbor Embedding (TSNE) algorithm is another method for dimension reduction better suited for data visualization specifically, as it avoids the crowding problem often seen with PCA. However, that enhanced interpretability comes at a cost, as information loss is generally greater for TSNE over PCA. In this video, Mr. Meredith shows how this algorithm can be applied and visualized, contrasting alignment of Doc2Vec output with GICS sectors against that of PCA vectors.

Video Transcript

For data visualization purposes, we want to introduce a second method of dimension reduction called the TSNE. It's the t-distributed stochastic neighbor embedding. It's a mouthful, so we just go with the TSNE.

TSNE uses a different mathematical method for dimension reduction. We talked about how PCA goes through and maximizes the variance that it's going to capture so mathematically it's preserving as much information as possible while you're compressing the dimensions.

TSNE is a little more appropriate for data visualization because it does the dimension reduction in two stages. First, it constructs a probability distribution over pairs of high-dimensional objects in such a way that you're essentially creating for things that look pretty close to one another a higher amount of distance.

Then within it, it's going to do a probability distribution where it's going to in the low-dimensional minimize the diversion. It actually uses the **kL** divergence to go through and say, let's push these distributions a little more together. What you wind up with while it's still preserving information and reducing dimensions, it can be really good for data visualization.

What happens is, you saw in that last algorithm, there's a part where PCA is basically maximizing the information preservation. But it can lead to a problem called the crowding problem, where in data visualization it can have it where it's going to have a center mass where everything's going to crowd against it. What's going to happen



with the TSNE is it's going to allow you to maximize that first pass where it's going to preserve some of the distances within the groups. Then it's going to try to minimize the distance and see how similar these are within them.

With that, let's walk through the code, and I'll show you how easy it is again to do one of these TSNE algorithms. When we dive in, from `sklearn.manifold` we're going to `import TSNE`. None of the math we have to do. We borrow the open source, utilize the algorithm, and this is incredibly straightforward. `TSNE`, we're going to do number of components, three. It's at a random state. We're going to `model.fit_transform` that vector again, and we're going to print it out, and this will look familiar. Another set of vectors where it's coming through — in this case it looks like a higher range in the -9.28 for Apple, -2.5 and a 4.36. You can see where assigning those again back to our dataframe where we're going to have an XYZ plot that we're going to put back into our `plotly` framework to try to evaluate how clean this model is at preserving the information that's coming through our business descriptions and how closely aligned that is with the GICS groupings.

By the way, checking the vectors again just to make sure this lines up with what you would expect. You always want to make sure that you're not by accident transforming your data and doing the merging. I mentioned about the resetting of the index in an earlier lesson and making sure that that's aligned. That's problem I ran into. So -9.28 lines up with my -9.28, -2.58 there, 4.36, 4.36. And we're good.

In this case we're going to redo our `plotly.express`. We're going to do that same `px.scatter_3d`, same dataframe, same color scheme, hover-over name, log market cap, same size and everything. But in this case the XYZ, it'll be our TSNE, X, Y, and Z. Same outlines. Going to write a different HTML file so we can have that separate in case you want to pull it up inside of that framework.

We're going to show the figure. In this case, what you see is a cleaner — remember the crowding problem we saw within the other algorithm, where everything was hovering around the center? In this case, much cleaner representation of groups. In this case our pharmaceuticals, you can see a clear space and alignment between these and how they are outside on their own little grouping.

You can see over here that sense of these consumer, discretionary, and staples, there's a bit of a grouping between those two, the lighter purple and the darker purple. That's



our discretionary versus our staples, so National Beverage at a staple at a 302010. Having it where our City Trends or Tilly's more of a consumer discretionary. You can see down here our materials and our energy companies — our hover over is a little tough on this sometimes — but like Range Resources, Callon Petroleum, etc., those are getting a little bit closer.

Our utilities group is going to wind up closely aligned with our materials and our energy. It makes sense. Our real estate companies have a grouping in here where we can see those together. A couple of outliers. WeWork being one, Newmark another. There's a couple in here that are actually aligned with the tech space. Zillow Group makes sense, where that's going to be a little bit different. There were a couple others that I was seeing previously that were going to be names that might be a little bit separate from the traditional real estate.

Of course, then here financials having its own distinct group on this side. The technology over in here you can see it has its own grouping as well, closer on the health care. But again, this visualization allows us to come back, try to give an evaluation of the model, and saying, how well is Doc2Vec doing and pulling out the semantic meanings from the text? How closely is that aligned with an external validation metric like our GICS? This is telling me that our doc2vec is doing a pretty good job pulling out the meanings from the business descriptions. It seems like it's going to be pretty highly aligned with what's going on inside of the GICS descriptions.

[Back to Table of Contents](#)



Code: Implementing TSNE

This is an ungraded practice activity.

PCA may be an effective method for dimension reduction, given the linear methods it employs to minimize information loss and maximize variance in reduced dimensions. However, it is not necessarily ideal for data visualization, as you have seen. TSNE, on the other hand, uses an entirely different mathematical process for dimension reduction that can enhance data visualization, producing cleaner visual groupings for analysis. The downside is that TSNE also has a greater potential for information loss and false relationships over PCA. For that reason, PCA is recommended more for dimension reduction in feature engineering, whereas TSNE is recommended for data visualization specifically.

In this activity, you will use the TSNE algorithm to reduce the dimensions of Doc2Vec loadings from 150 to 3, allowing you to use Plotly for visualization of semantic vectors in three dimensions. You will then evaluate alignment of Doc2Vec with GICS to determine the relative efficacy and validity of TSNE over PCA in associating companies with topics.

Activity Instructions: Using the sandbox below, address the following tasks/questions to reduce the loadings of the Doc2Vec model from 150 dimensions (loadings) to 3 with TSNE, then use Plotly to visualize the resulting vectors of the Top 1000 companies. Once you feel confident in your response, click the **Solution** button located beneath the sandbox to reveal the answers.

1. Run `sklearn.manifold.TSNE` on `d2v` vectors to reduce their dimensions (loadings) from 150 dimensions to 3.
2. Audit those loadings to verify reduction to 3 dimensions.
 - List loadings for the vectors of the top 10 companies by Market Cap.
3. Run Plotly to build a 3D scatter plot of the `d2v` vectors.
 - Can you see some separation between GICS groups in the chart?
 - Is there some crowding around the center?
 - How effectively do TSNE reduced Doc2Vec loadings capture the underlying meaning of business descriptions?



Please access this Jupyter Notebook exercise in the course.

Solution



2. Audit those loadings to verify reduction to 3 dimensions.

a. List TSNE vector loadings of the top 10 companies by market cap.

Answer: Apple (x: -3.211, y: 2.212, z: -2.117); Microsoft (x: -0.554, y: 5.058, z: -5.505); Alphabet (x: -2.985, y: 3.094, z: -2.457); Amazon.com (x: -3.062, y: 2.055, z: -1.889); Tesla (x: 1.151, y: -3.454, z: -5.667); Meta Platforms (x: -3.021, y: 3.753, z: -2.556); NVidia (x: 5.467, y: 2.327, z: -6.561); Berkshire Hathaway (x: -0.568, y: 3.388, z: 8.792); UnitedHealth Group (x: 4.296, y: 5.237, z: 3.497); JPMorgan Chase (x: -3.379, y: 8.800, z: 5.277).

3. Run Plotly to build a 3D scatter plot of the **d2v** vectors.

a. Can you see some separation between GICS groups in the chart?

Answer: Yes, when rotating the chart you can see a clear separation by GICS color schemes, especially so for Health Care, Financials, Utilities, and Energy. Groupings are significantly more distinct and separated than they were for the PCA visualization.

b. Is there some crowding around the center?

Answer: Yes. There is some crowding around the center, but far less than there was for PCA. This implies that dimension reduction via TSNE was mostly effective from a visual perspective in separating companies by industry or activity type.

c. How effectively do TSNE reduced Doc2Vec loadings capture the underlying meaning of business descriptions?

Answer: While company groupings are even more distinct for TSNE, the inferences from the PCA data visualization hold. The reduced dimensions for Doc2Vec loadings provide clear visual evidence that the underlying meaning of business description text is being captured effectively. The proximity of companies with the same GICS classification (Financials, Health Care, Utilities, Energy) means that the business activities of companies are accurately reflected in the continuous numerical landscape. It also implies that any outliers may be interesting to look into, as their current GICS classification may not truly reflect their business activities.

[Back to Table of Contents](#)



Module 4.1 Assignment: Training and Evaluating a Doc2Vec Model

As you have seen, the Doc2Vec algorithm preserves the order of the words to generate semantic vectors, drawing associations between companies by topic similarity rather than word frequency as the LDA algorithm did. However, unlike LDA, Doc2Vec output can be significantly challenging to interpret.

Fortunately, given the continuous numerical space generated by Doc2Vec, similar documents and/or words can be mathematically reduced, which helps to both distinguish similarities between companies and visualize those relationships through the incorporation of dimension reduction algorithms like PCA and TSNE.

In this assignment, you will perform the same operations that you did in previous activities for the Top 1000 companies, training a Doc2Vec model on the business description corpus of the Middle 1000 companies and evaluating for alignment against GICS via PCA and TSNE reduced visualizations of model output.

Completion of this assignment is a course requirement.

Instructions:

1. Download the **Module 4.1 Assignment** template to use with the sandbox below.
2. Train your first Doc2Vec model on the cleaned business descriptions from Unit 2, merging the resulting loadings to create vectors specific to each of the Middle 1000 companies. Then reduce the loadings of the Doc2Vec model from 150 dimensions (loadings) to 3 with both PCA and TSNE to visualize and compare the resulting vectors against GICS with Plotly.
3. Use the output to address tasks present in the assignment template.
4. Save your work locally as an accepted file type (.doc, .docx, .txt, .pdf) with the following naming convention:
BANA5210_LastName,FirstName_Module4.1Assignment. No other file types will be accepted for submission.
5. Once you are confident in your responses, click the **Start Assignment** button to upload and submit that file for manual review.



Before you begin:

Please consider that this is a professional forum; courtesy and professional language and tone are expected. When you submit work for academic credit, you are certifying that the work is your own. All outside assistance should be acknowledged, and your original academic work and ideas truthfully reported at all times. Whether intentional or unintentional, plagiarism is a serious violation of **Johnson's Honor Code** and **Cornell's Code of Academic Integrity**. Avoid plagiarism by doing original work, and always citing, acknowledging, and giving credit to all sourced materials.

Please access this Jupyter Notebook exercise in the course.

[Back to Table of Contents](#)



Module Wrap-up: **Train and Evaluate a Doc2Vec Model**

In this module, you prepared data for the Doc2Vec algorithm and trained your first model, converting business descriptions into numerical, semantic vectors for analysis. You then mathematically reduced the dimensions of Doc2Vec output via PCA and TSNE, comparing and contrasting their semantic relevance to company and sector assignments. Finally, in the module assignment, you trained and evaluated a Doc2Vec model for the business description corpus of the Middle 1000 companies, incorporating dimension reduction to produce two different data visualizations for evaluation of model performance and alignment with GICS.

[Back to Table of Contents](#)



Module Introduction: Use Doc2Vec Loadings to Find Comparable Companies



Another useful application of Doc2Vec is in building lists of comparable companies which, unlike LDA, already draws contextual, semantic correlations between output vectors, rather than relying on word frequencies. That difference is critical, as it not only allows for semantic comparison of companies, but for words as well, which can be leveraged to perform a number of different operations relevant to the construction and management of investment portfolios.

In this module, you will build lists of comparable companies by similarity in semantic meaning, evaluating the relevance of activity associations made against GICS. You will then build lists of words comparable to single and combined vectors to explore the mechanism underlying Doc2Vec semantic associations, customizing your own semantic vectors to build lists of companies comparable to a particular theme. The module assignment asks you to identify words in a business description corpus relevant to two specific investment themes, incorporate them into custom semantic vectors, and build lists of companies comparable to those themes specifically.

[Back to Table of Contents](#)



Watch: Use Distance Metrics to Find Comparable Companies Part 1

As you may recall, Hellinger distance was required to build lists of comparable companies with the LDA algorithm, with calculated distances used to align companies by topic similarity. Given the difference in input for Doc2Vec, that same alignment is determined by similarity in semantic meaning rather than topic. In this video, Mr. Meredith demonstrates application of a Doc2Vec function purpose built for this operation, comparing results against GICS to highlight relevant differences.

Video Transcript

We want to start utilizing the Doc2Vec algorithm to find the competitive companies similar to what we did with the LDA algorithm in the past. Couple of things to remember. If you remember the LDA algorithm, we hit home on that point where what we're doing is we're generating those topic loadings and then having it where the percentage that you've included in that topic, we're looking at the similarity of that. In that case we used Hellinger distance to figure out how similar the topics would be. So if you're 60% inside of retail, 40% inside of food, that would put you as a fast food restaurant potentially and finding other ones within that category. But it was first classifying you into categories, and then looking for the percentage inclusion in that category to find the similar company.

In this case, we're going to use the Doc2Vec loadings that are coming from that semantic meaning of the vectors, creating 150 dimensions, and we're going to go through and try to figure out how similar the companies are. Now the difference of this is rather than going off the similarity by topic it's going to be the similarity by semantic meaning, which can be a benefit for this.

In particular, if you think about what's going on with, in this case, we'll start with Tesla. First of all, we'll go through this. We'll use a little bit of a difference. Rather than define mathematically the similarity matrix ourselves, we're just going to borrow gensim's matrix similarities, which is actually a cosine similarity. It's just a mathematical technique of determining the distance, but it's a similar way of going about and doing that.



From that, we're going to have our `idx`, which will be we're going to take our `d2v` dataframe where we'll take the index, we'll pull out the Tesla and put it into a list. We're going to find the query, this vector that's got the index from it. This will be a way just to pull out from the vector by itself numerically which row of that 150 loadings are we pulling out? We're going to then take that `vec`, create a `MatrixSimilarity`, and we're going to push that in as the `sims` with a `matrixsim[query]`. We're going to sort on that and take the `sims`, do the `key=lambda item: -item`, and that's going to give us out what the similarity matrix, which one will be the closest to it.

For that, we'll get the most similar companies. In this case, we're going to go through — it's going to be a little bit of a different way that we're referencing the dataframes because we're going to be going off of this `compindex`, and there'll be a way of saying, give me the one that's going to be the one referenced inside of it. We're going to create the most similar companies off of this. We're going to do the top 10, so range 0 to 11 — really top 11 because we're including itself in this. So the very first one, or close to it, first or second, should be the company itself.

We're also going to print out the business descriptions to show what those are for these companies, to show how similar we think it is to the company itself, and then we're going to do the companies in the subindustry to show the difference between what's inside of this grouping that we're doing versus — remember, we were talking about finding competitors. Sometimes the best thing you can run off of is just what's inside of the subindustry.

In this case, for `Tesla` what we see is our most similar companies, `Tesla` being number one. We then have `Proterra`, `Volta`, `Toro` — so Tesla wound up being number two, again, highly similar — `Volta`, `Toro`, `Hyliion Holdings`, `Capstone`, `Green Energy`, `Hyzon Motors`, `KVN Industries`, `American Superconductor`, `Polaris`, and `Generac Holdings`.

If you start reading these, "Proterra provides commercial vehicles in the United States. It operates through Proterra Powered and Energy and Proterra Transit business units. It also integrates battery systems, electrification solutions into vehicles." This is highly aligned. Same thing, Volta, "network of smart media-enabled charging stations for electric vehicles in the United States." Pretty decent. Toro was an interesting one. It's engaging in essentially "professional residential equipment worldwide." I don't have



the full description, but I'm assuming there's something in there for vehicles and electrical will wind up coming through, and battery charging.

But other ones here are **Hyliion**. We'll have "commercial vehicle industry, cells, electrified powertrain solutions." **Capstone**, "microturbine technology solutions." **Hyzon Motors**, "decarbonized solutions for commercial vehicles and hydrogen gas supply." These are all basically aligned with alternative energy, battery powered systems, and vehicles overall.

If you look at the subindustry for Tesla, what you're seeing here is **Tesla** essentially has it where it's in the 25102010 as cars. Its main competitors when you look at that are **General Motors**, **Ford**, **Lucid Group**, **Thor Industries**, **Winnebago**.

Looking at various companies inside of this, not as highly aligned. In this case, the doc2vec is giving us that targeted semantic meaning for those components of vehicle, electric, battery, systems, etc., while the GICS subindustry is just thinking about automobiles.

[Back to Table of Contents](#)



Watch: Use Distance Metrics to Find Comparable Companies Part 2

Similar to LDA topic associations, Doc2Vec can be significantly more flexible in its semantic associations over GICS. To get a sense of those differences, consider the extent to which GICS subindustries limit perceived alignment of business activity. In this video, Mr. Meredith uses the semantic vector framework to build comparable companies lists for a variety of industries, contrasting their relevance to activity alignment against lists for equivalent GICS subindustries.

Video Transcript

In the last video, we showed you how we use this semantic vector framework to get comparable companies to Tesla and how we're able to show companies that were much more in the electric vehicle battery space versus just straight-up automobiles. We're going to repeat that exercise to show the power of the framework and some of the other challenges you may run into when trying to build a competitive company list.

We're going to start with **Equinix**. Equinix is a data center, but it's classified as a REIT because it holds a lot of real estate holdings. So this is one of those cases where because of the real estate nature of it, it's getting classified by GICS along with all of the other physical asset companies, but the real primary business of this company is about cloud services.

So when we run through **EQIX** in the ticker, running through the same bit of code, what you can see is that from our semantic vector framework, **doc2vec**, **Equinix** gets **KVH Industrials**, **Calix**, **Casa Systems**, **Palo Alto**.

When we walk through the descriptions of itself, you can see **KVH** is basically "mobile connectivity products and services for land, mobile." **Calix** winds up being "cloud and software platforms in the United State." **Casa** — "next-generation physical virtualized cloud native architectures for cable broadband," cybersecurity through **Palo Alto**, etc. What you get a sense for is that the companies that are coming through may have some real estate, but it's also going to have a lot of the cloud computing component,



which is a key part of its business, versus when you scroll down and you say, what's the number one in the subindustry?

This is sorted by market cap, so it's the largest, but it's **Crown Castle**. It "leases more than 40,000 cell towers and approximately 80,000 route miles of fiber." It's more of a physical infrastructure than cloud computing. That's the number one on market cap, where **Equinix** is the number two. **SBA Communications** is "leading owner and operator of wireless communication infrastructure." So again, another in tenant space, multi-tenant communication site. From the example that we're showing here, **EQIX**, even though it's physical assets, it's mainly in the cloud computing space, and that's where our algorithm is able to pick that up.

We're going to do a similar exercise and go through it for in this case **XOM**. The reason for this is that **Exxon Mobil** — I'll show you at the very bottom of this description — has only four companies, three other competitors, that's inside of its subindustry. So when you're trying to build a list, you're going to get **Exxon Mobil**, **Chevron**, **Occidental Petroleum**, and **Unit Corp**.

So in this case, while they're all integrated EMP, energy and oil production, companies, you can have it where you may be lost and saying, well, I'd like to have some more companies to look at, in which case then our Doc2Vec framework is going to add to it. So we're going to have here things like **Kosmos Energy** on top of **Chevron** and **ConocoPhillips**, we get the **Freeport McMoran**, **Newmont**, **Phillips 66**, **Matador Resources**. We're getting some materials as well as some energy companies inside of it, and that's because what we have here is the full business description where it's petroleum, petrochemicals, olyphenols, Polyphenols, aromatics.

So Exxon, even though it's going to be in that crude oil and natural gas, is getting into some of the materials companies as well, that idea of utilizing the oil to build things out and petrochemicals. That's where you're seeing Kosmos coming through as an oil and gas company. You're going to get, in this case, other companies down here that are going to fit more into the materials company as well. So that gives you more companies. Again, this is the main part of the exercise, which is **XOM** has a narrow list from the subindustry. Utilize the model to build yourself a broader list.

On the flip side, **Netflix** has the opposite problem. When we look at **Netflix**, what we see is that in its subindustry, the subindustry is fairly broad. So we're scrolling down.



This is where we're seeing within the subindustry **Disney**, **Netflix**, **Roku**, **Live Nation**, etc. But you're dealing with a list of 25-plus names, and you say, well maybe I want to know which ones are the most closely aligned.

This is where the semantic vectors help. You can know that **Netflix** — from the business description and all the words that are inside of that with "TV series, documentaries, feature film...the ability to receive streaming content through a host of internet-connected screens, TVs, digital video players," etc. You can wind up seeing that the next one, **ViacomCBS**, where we get **Paramount Global** as part of this. We can get the **Madison Square Garden Entertainment** is the closest to it. **AMC Networks** is the closest to it. **Walt Disney**. You can narrow down your list. Instead of starting with 25 and not knowing how to sort it, you can use this to basically say, give me the five closest. That's where you can take these top five and use that to start doing your comparative and competitive analysis.

[Back to Table of Contents](#)



Code: Using Distance Metrics to Find Comparable Companies

This is an ungraded practice activity.

Hellinger distance was used as a metric to build lists of comparable companies based on LDA output, with distance calculations reflecting topic similarity rather than word frequency associations. Given that the Doc2Vec algorithm already generates topics and aligns companies by semantic meaning, building a comparables lists requires significantly fewer steps.

In this activity, you will utilize the semantic vector framework to build a list of comparable companies for the business description corpus of the Top 1000 companies, evaluating their relevance to activity alignment against lists for equivalent GICS subindustries.

Activity Instructions: Using the sandbox below, address the following tasks/questions to run the `d2v.docvecs.most_similar` for Tesla, Amazon and Exxon Mobil to build a list of comparable companies for the Top 1000 companies and evaluate their relevance against GICS sub industry groupings. Once you feel confident in your response, click the **Solution** button located beneath the sandbox to reveal the answers.

1. Run `d2v.docvecs.most_similar` for Tesla.
 - List the top 10 companies nearest `TSLA`.
 - List the companies in the same GICS sub industry as `TSLA`.
 - How do those two lists compare with respect to the identification of comparable business activities?
2. Run `d2v.docvecs.most_similar` for Amazon.
 - List the top 10 companies nearest `AMZN`.
 - List the companies in the same GICS sub industry as `AMZN`.
 - How do those two lists compare with respect to the identification of comparable business activities?
3. Run `d2v.docvecs.most_similar` for Exxon Mobil.
 - List the top 10 companies nearest `XOM`.
 - List the companies in the same GICS sub industry as `XOM`.



- How do those two lists compare with respect to the identification of comparable business activities?

Please access this Jupyter Notebook exercise in the course.

Solution



1. Run `d2v.docvecs.most_similar` for Tesla.

a. List the top 10 companies nearest `TSLA`.

Answer: Tesla 25102010; Generac Holdings 20104010; Borgwarner 25101010; Enphase Energy 45301010; General Motors 25102010; Sensata Technologies 20104010; Wabtec 20106010; Fisker 25102010; Chargepoint Holdings 20104010; Rivian Automotive 25102010; Lear 25101010.

b. List the companies in the same GICS sub industry as `TSLA`.

Answer: 25102010 - Tesla, Rivian Automotive, General Motors, Ford Motor, Lucid Group, Thor Industries, Fisker.

c. How do those two lists compare with respect to the identification of comparable business activities?

Answer: The GICS classification is 25102010, which represents automobile manufacturers. This list includes more traditional companies like General Motors and Ford, as well as more closely aligned electric vehicle makers Rivian, Lucid, and Fisker. Thor, on the other hand, is an RV manufacturer. The Doc2Vec model does recognize General Motors, Rivian and Fisker as similar companies, but is also aligning Tesla with battery technology companies like Generac, BorgWarner, Enphase and Chargepoint, which are good additions as a significant portion of Tesla's business involves home power storage solutions like the Powerwall. Sensata, Westinghouse and Lear seem more like industrial components, making their reasons for being comparable less readily apparent.

2. Run `d2v.docvecs.most_similar` for Amazon.

a. List the top 10 companies nearest `AMZN`.

Answer: Amazon.com 25502020; Activision Blizzard 50202020; Digital Turbine 45103010; Gamestop 25504020; Alphabet 50203010; Electronic Arts 50202020; Apple 45202030; Zynga 50202020; Roku 50202010; New York Times 50201040.

b. List the companies in the same GICS sub industry as `AMZN`.

Answer: 25502020 - Amazon.com, Doordash, EBay, Etsy, Chewy, Wayfair.

c. How do those two lists compare with respect to the identification of comparable business activities?

Answer: Amazon is classified under 25502020 - Internet & Direct Marketing Retail, which places it alongside a handful of other internet retail sites. The Doc2Vec model, on the other hand, includes a range of different companies. Alphabet is a communication service that also has cloud hosting capabilities like AWS. Roku is a streaming platform. Activision, Zynga, and Gamestop are



gaming companies. The New York Times is a traditional news company. And finally, Apple and Digital Turbine are mobile platforms for advertisers and publishers. This shows that Doc2Vec is capturing some of the other business activities that Amazon has in addition to online retailing.

3. Run `d2v.docvecs.most_similar` for `XOM`.

a. List the top 10 companies nearest `XOM`.

Answer: Exxon Mobil 10102010; Chevron 10102010; EOG Resources 10102020; Chesapeake Energy 10102020; Hess 10102020; ConocoPhillips 10102020; Pioneer Natural Resources 10102020, Ovintiv 10102020; APA 10102020; Marathon Oil 10102020; Devon Energy 10102020.

b. List the companies in the same GICS sub industry as `XOM`.

Answer: 10102010 - Exxon Mobil, Chevron, Occidental Petroleum.

c. How do those two lists compare with respect to the identification of comparable business activities?

Answer: Exxon is one of three 10102010 Integrated Oil & Gas companies that are in the top 1000 stocks by market cap. The list of comparable companies through Doc2Vec, however, has shifted to include companies from GICS 10102020, Oil & Gas Exploration & Production. This indicates that there is more activity alignment for Exxon Mobil with exploration and production than simply selling/providing oil and gas.

[Back to Table of Contents](#)



Discuss: Doc2Vec vs. GICS vs. LDA

Discussion topic

As you have seen, lists of comparable companies can be built specific to GICS subindustries, by topic similarity with the LDA algorithm, or by similarity in semantic meaning with the Doc2Vec algorithm. While these methods each have their own unique strengths, there is a significant difference in the correlations drawn between companies as a result of their alternative approaches to the development of comparable lists.

Given your experience at this point, do you feel that there are areas where one method is preferable to another in the construction and management of an investment portfolio? While GICS is still maintained as an industry standard against which comparable company lists are built to this day, there are restrictions to those classifications that limit their applicability. However, while LDA and Doc2Vec may offer improvements over GICS in the relevance of correlations drawn between companies, their respective efficacy is limited by the relevance of input data.

Your exploration so far may have provided you with ample evidence of the methods' overall alignment to one another, but it is important that you consider their respective differences before applying any of them in the real world. Take a moment to share your thoughts on the topic with your classmates, structuring your response around the following question:

How might you determine which method to employ when building a list of comparable companies for the construction and management of an investment portfolio?

- Areas of strong alignment between Doc2Vec/LDA and GICS.
- Areas where Doc2Vec is superior.
- Areas where LDA is superior.
- Areas where GICS is superior.
- Which method would you choose given your own investment interests?



Instructions

You are required to participate in all course discussions. To be able to see and respond to the posts of your peers you must post a response. Keep your initial posts focused and brief; no more than 200 words. Actively participate by asking questions of your peers and responding to their posts with your own comments. *To receive full credit you are only required to post an initial response to the discussion prompt, although it is highly recommended that you also reply to the posts of at least two of your peers.*

To participate in this discussion: Click Reply to post a comment or reply to another comment.

Your initial post will be assessed according to the following criteria:

- Comprehension - *Demonstrates an understanding of course content.*
- Synthesis - *Demonstrates ability to synthesize course content within the discussion topic.*
- Originality - *Utilizes original thinking.*
- Development - *Clearly articulates and fully addresses all requirements.*

This is a professional forum; courtesy and professional language and tone are expected. When you submit work for academic credit, you are certifying that the work is your own. All outside assistance should be acknowledged, and your original academic work and ideas truthfully reported at all times. Whether intentional or unintentional, plagiarism is a serious violation of **Johnson's Honor Code** and **Cornell's Code of Academic Integrity**. Avoid plagiarism by doing original work, and always citing, acknowledging, and giving credit to all sourced materials.

[Back to Table of Contents](#)



Watch: Use Doc2Vec to Find Similar Words through Semantic Vectors

Another advantage provided by the vectorization of Doc2Vec input is the ability to find words similar to a desired word of interest. By applying semantic word vectors to single words and entire documents, you can gain insight into how those vectors are built. In this video, Mr. Meredith goes over how to build lists of comparable words for both single and combined semantic vectors, pointing out the mechanism behind their construction and implications for their use.

Video Transcript

The beauty of the Doc2Vec framework is that you get the semantic word vectors and you can apply those to the companies which have the entire document and corpus attached to it or you can apply it to single words and use them to try to build up to — in that mathematical framework, going through and identifying companies from the ground-up foundation of using vocabulary in itself.

This is interesting because first, we'll go through and see how the semantic vectors from words individually are built up, similar to genetic. We can say `doc2vec`, give me the most similar words to genetic, in this case fashion, or restaurant, or oil, or finance, social media, or plant. What you can see is `doc2vec` will spit back very clearly, similar to `genetic`, we're getting the words `molecular, whole_genome, gene_expression, genome, hereditary, exome, genomic_sequencing, genotyping, diagnostic_tests`. It's understanding that frequency of words in the sense of probabilistic distribution and saying, when I see this word, I would expect this word near it. It's saying for `genetic`, words that are going to be most similarly used to that is going to be this list of 10.

Similar on `fashion`: `apparel, accessories, sportswear, footwear_accessories, men, women, shoe, dress, clothing, apparel, lifestyle, and handbag`. It's pretty apparent `fashion` is coming through with that semantic sense of having to deal with things like `sportswear, footwear, handbags, clothing`, all coming through and getting the same meaning as `fashion`.

For restaurants — let's go to the next one — `casual_dining, fast_casual, cafeteria, diner, luby, cabana, restaurant_chain`, these may be names that are attached to it, `Applebee's`.



certainly being one, `dine`, `taco`. All ways of saying these parts are attached to what the restaurant meaning would be.

For `oil`: `petroleum, distillate, refiner, refining, blending, biodiesel, gasoline, diesel`, again, getting clean out of that. For `finance`: `financing, leasing, liquidation, floor_plan, originator, pension_fund`, attaching to `finances` is pretty clean.

`social_media`: there's `search_engine, advertising, campaign, publisher, influencers, campaign, advertising, advertiser, advertisement, Instagram`.

And then `plant`. `Plant` is an interesting one because if you remember back in the word frequency, were talking about the multiple potential meanings for it and how it was classified across multiple topics because it was split up between agriculture and then energy because you're going to wind up having agricultural plants and then steam and oil plants. This is one where you can see it come through very similar to what's going on inside of our `doc2vec`, where `biomass` and `biogas`, more of the agricultural, but `steam, co-generation, methane, byproduct, coal_fired, fermentation, coal`. We're getting both of those semantic senses inside of the word frequency.

It's important to note words that may have multiple meanings could potentially give you multiple semantic vectors that are coming out of it. Also, because of the mathematical framework, we can start to combine these and put them together. In the case of this, we can put, give me the most similar to the word vector for `energy` and the word vector for `clean`. What you get out is first `energy, clean, renewable_energy, renewable_wind, solar, hydro`. You're still getting some `steam, renewables`. All of these are ways of saying when we're going to get clean energy, these are words that are more aligned with that addition.

Again, you can have it where you're going to take one semantic vector, apply another semantic vector, and the final output mathematically would be something more closely aligned with what you're looking for. Same thing for `virus` plus `vaccine`. You're getting closer to this part of `influenza, sars, covid, viral, zika, coronavirus, hepatitis b`, etc.

But this framework of having it where you can mathematically combine semantic vectors is highly powerful. We're going to show you how to utilize this to build up lists of comparable companies, not from having one company and then building out a list,



but starting raw from the words that you want to utilize to build your list of competitive companies.

[Back to Table of Contents](#)



Code: Finding Comparable Words

This is an ungraded practice activity.

As you have observed, another benefit of the vectorization of Doc2Vec input is the ability to find words similar to a desired word of interest, which provides insight into how semantic vectors for companies are constructed by the algorithm. By building lists of comparable words through this method, you will eventually be able to customize semantic vectors specific to your own desired theme, rather than relying on default associations provided by the algorithm.

In this activity, you will apply the Doc2Vec `most_similar` function to find words closest to a set of single and combined semantic vectors within the business description corpus of the Top 1000 companies, comparing the relevance of associated words to your initial inputs.

Activity Instructions: Using the sandbox below, address the following tasks and questions to run the `d2v.wv.most_similar` for specific words to build a list of comparable words for the Top 1000 companies and evaluate their relevance to the original input(s). Once you feel confident in your response, click the **Solution** button located beneath the sandbox to reveal the answers.

1. Run `d2v.wv.most_similar` for `tissue`.
 - List the top 10 most similar words identified.
 - How is `d2v` identifying the word `tissue`?
 - Is this list relevant?
2. Run `d2v.wv.most_similar` for `plant`.
 - List the top 10 most similar words identified.
 - How is `d2v` identifying the word `plant`?
 - Is this list relevant?
3. Run `d2v.wv.most_similar` for `genetic` and `oncology`, both individually and as a combined loading.
 - List the top 10 most similar words identified for `genetic`.
 - List the top 10 most similar words identified for `oncology`.
 - List the top 10 most similar words identified for `genetic` + `oncology`.



- How do the lists differ when these words are run as separate vs. combined loadings?

Please access this Jupyter Notebook exercise in the course.

Solution



1. Run `d2v.wv.most_similar` for `tissue`.

a. List the top 10 most similar words identified.

Answer: `regenerative` (0.974), `electrophysiology` (0.963), `blood` (0.962), `skeletal` (0.956), `deformity` (0.956), `implantable` (0.955), `respiratory` (0.955), `urine` (0.952), `degenerative` (0.952), `prostate` (0.948).

b. How is `d2v` identifying the word `tissue`?

Answer: `d2v` is identifying the word `tissue` as human tissue, rather than tissue paper.

c. Is this list relevant?

Answer: It is relevant for companies that are focused on health care. However, there is potential for noise when a company has a specific focus on tissue paper, a Consumer Products company for example.

2. Run `d2v.wv.most_similar` for `plant`.

a. List the top 10 most similar words identified.

Answer: `underground` (0.930), `coke` (0.922), `metallurgical` (0.921), `gallon` (0.917), `aquifer` (0.917), `consumes` (0.915), `million_tons` (0.911), `steelmaking` (0.910), `diesel_fuel` (0.910), `coal` (0.907).

b. How is `d2v` identifying the word `plant`?

Answer: `d2v` is identifying the word `plant` as power plant, rather than a biological plant.

c. Is this list relevant?

Answer: There is the potential for association with industrial, utility, or energy companies if a company's business is focused on industrial plants. However, there is a potential for noise when a company has a specific focus on biological plants.

3. Run `d2v.wv.most_similar` for `genetic` and `oncology`, both individually and as a combined loading.

a. List the top 10 most similar words identified for `genetic`.

Answer: `genomic` (0.988), `translational` (0.977), `genotyping` (0.975), `gene` (0.975), `dna` (0.974), `biomarkers` (0.967), `profiling` (0.965), `exome` (0.963), `infectious_disease` (0.962), `genome` (0.962).

b. List the top 10 most similar words identified for `oncology`.



Answer: `therapeutic_areas` (0.982), `neurology` (0.973), `medicine` (0.971), `oncologist` (0.958), `disease` (0.957), `immunology` (0.956), `transplant` (0.953), `neuroscience` (0.951), `therapy` (0.950), `theralink` (0.942).

- c. List the top 10 most similar words identified for `genetic` + `oncology`.

Answer: `therapeutic_areas` (0.976), `infectious_disease` (0.976), `oncology` (0.974), `genetic` (0.973), `neurology` (0.969), `transplant` (0.966), `genomic` (0.963), `disease` (0.962), `guardant` (0.961), `biomarkers` (0.958).

- d. How do the lists differ when these words are run as separate vs. combined loadings?

Answer: `therapeutic_areas` remains the most relevant word, but `infectious_disease` moves up in the list, as both are more relevant for the combined loading than for `oncology` and `genetic` on their own. `guardant`, however, is a new list addition, and makes sense because Guardant Health is a company that provides blood tests for oncology through genome analysis. As separate loadings it would make sense why `guardant` might have been further down, but as a combined loading, `guardant` becomes significantly more relevant.

[Back to Table of Contents](#)



Watch: Use Doc2Vec to Find Companies Aligned with Semantic Vectors — BOTZ

Thematic portfolios are a growing area of the investment market. They are composed of companies within a particular theme, and they tend to be more diversified across sectors. In this video, Mr. Meredith demonstrates how semantic vectors can be used to replicate that strategy by building a list of companies aligned with a real-world exchange-traded fund (ETF).

Video Transcript

A growing area in the investment market is this idea of thematic portfolios. Thematic portfolios are where you're not investing in one sector or subindustry, but it's this overriding theme where you're going to invest in something that is potentially an innovative technology that will affect the world over the next 20 to 40 years.

There's some of these that are out there. I'll give you an example of one, which is robotics. Global X provides one called BOTZ, where it's Robotics & Artificial Intelligence ETF. With this, you can see there's a number of companies that they've identified as ones that apply to this theme, Intuitive Surgical, ABB, Keyence, Fanuc, NVIDIA, Omron, etc. This is where a fundamental analyst has gone through and built out a list of what these thematic companies might be.

What we're going to do is see if we can replicate what's coming out of that ETF and do it through our `doc2vec` algorithms. We're going to start here with a botz, we're going to call that, as the summary of word vectors for — starting with `robotics` and `artificial_intelligence`, we're going to build out a `MatrixSimilarity` from that vector — oh I get the vector from it. We're going to do the `MatrixSimilarity` to `botz`. We're going to get the `sims`, which is the similarities. We're going to sort it, enumerate it, and go through it. We're going to get that range of companies, and we're going to build it out.

The very first pass we get is we start with `Ironnet`, `Adobe` — a little counterintuitive — `ANSYS`, `Palo Alto Systems`, `Northrop Grumman`, `NVIDIA`, and we can start coming down. Ironnet winds up being more of a cyberattack platform, `Adobe`, more of a diversified software company, `ANSYS`, are engineering and simulation softwares, potentially some of that firewall on this. `Northrop Grumman` is more of a defense company.



Let's see if we can try to refine this. What we're going to do is print out some words and say, what's similar to `robot`, `robotic`? We're going to see words like `ultrasound`, `robot`, `robotics`, `guided`, `sortation`, etc. We're going to continue to look through this and see what are the word frequencies in the words, `robot`, `AI`, `automate`, `autonomous` ...go through and evaluate this. Then what we're going to do is as you examine some of these things you're going to realize similar to `robotic`, in this case, we see `robot`, `robotics`, going through, computer aided, potentially there's other ones. We can extend what's going on inside of our word vectors.

In this case, I'm going to use `robotic`, `robotics`, `robot`, because we have different words that can then collapse down to the same semantic meaning, the sense for it. We're also going to add in things like `artificial_intelligence` and `AI` because that seems to be an area that's important for the robotic world, `automation`, `automating`, `factory`, `automating`, `automation`, `autonomous`, `autonomous vehicles`, `autonomous driving`, because that tends to be part of what's going on with the robotics.

By the way, part of the cheat on this, as we go back and get the overview you can look through the description and pull out words from this, where the Global X artificial intelligence seeks to invest in companies that potentially stand from increased adoption of robotics, artificial intelligence, autonomous vehicles, automation, nonindustrial robots, etc. When we borrow that, we're going to refine our list down and we get a new list that comes through where `ANSYS` is the top name inside of it. `Masimo` is at the top.

You can start looking through and seeing that this is one where "engineering simulation software for power budget analysis," continue to work through this having where "automation solutions from Masimo Corporation," Calix, cloud, and platforms inside of that. These are where we're continuing to refine our list of words that are going to be closer on AI, robotics, etc., and use those to refine our list and get down to something that's approximating closer to what's going on with this Robotics & Thematic ETF.

[Back to Table of Contents](#)



Watch: Using Doc2Vec to Find Companies Aligned With Semantic Vectors — ARKG

Semantic vectors can be used to build lists of companies aligned with a particular theme. To fine-tune that list, however, you must identify and incorporate relevant words into those vectors. In the previous video, words similar to robotics and artificial intelligence were incorporated for alignment to the BOTZ ETF. In this video, Mr. Meredith repeats that exercise with a different ETF for a thematic portfolio focused on genetics.

Video Transcript

We're going to repeat that exercise with a different ETF to reinforce the power of the platform. What we're going to use this time is ARK Invest Genomic Revolution ETF. This is one where, if you look at the overview, it's basically focused on "companies that are expected to substantially benefit from extending and enhancing the quality of human and other life by incorporating technological and scientific developments in genomics in their business."

They give these things here, CRISPR, which is a technique for gene editing, targeted therapeutics, bioinformatics, molecular diagnostic, stem cells, agricultural biology. We're actually going to start off by pulling in four of those: genomics, CRISPR, bioinformatics, and stem cell. We're going to then build the `arkg`, which is our replication here for that as the word vectors that are combining all of those. We'll get the `MatrixSimilarity`, pull that out, sort it, and enumerate it. Then for each one we're going to print 50 companies that'll have that.

We can see `NanoString Technologies, SomaLogic, Intellia Therapeutics, Sema4 Holdings, Quidel, Bionano`. When we start going through you can see `NanoString` "sells tools for scientific and clinical research in the field of genomics and proteomics in America." We're seeing here `SemaLogic`, "a protein biomarker discovery and clinical diagnostics company." Intellia Therapeutics, development on the focus of therapeutics. We go through, you can see `CRISPR` down here has an associated word for it. `Sema4 Holdings`, doing business "Sema4 operate as health information that enhances diagnosis, treatment, and prevention of disease through data," etc.



In all of this you can see if we want to we can go back and compare because we're going to have the holdings that will come through on this. We can see different names that are coming through. And what we can do is try to compare ones like Exact, Ionis, CareDx, etc.

We're going to make this a little bit more refined. We're going to come back and say, give me what's similar to the four words that we pulled out of the description for `genomics`, `bioinformatics`, `CRISPR`, `stem cell`. You can start looking and saying, `genomics` — `genetics`, `molecular_diagnostics` potentially, pulling through this and seeing what you may want to put inside of it for `bioinformatics`, `proteomic`, I'm not even 100% sure what it means, but it's obviously very close to the `bioinformatics`. We have to figure out what these words — `DNA_sequencing`, etc.

You want to have some idea of the words that you're feeding back into it, not do it blindly. Make sure it's not noise, and have it where you are going to be incorporating it. But what you're able to do here is figure out words that may be similar and may reinforce the word vectors and semantic meaning. `CRISPR`, `base_editing`, `base_editors`, `gene_editing` over here, and `stem_cell` where you can wind up with things like `bone_marrow`, `cord`, `blood`, etc., and pushing those in it.

What we're going to do is take our `arkg`, take the original four that we had and add in for each one of the `genomics`. We're going to have in here `molecular_diagnostics`, `proteomic`, `gene_editing`, put in the `cord`, `blood`, and `bone marrow`, and run these together and see what we get out. Looking here, we've got our 10x genomics that's coming through, different names than the last time around and potentially more aligned with what we're trying to get to, which is having the descriptions of everything that's going to be in that field of "extending life through genomic and genetic research."

[Back to Table of Contents](#)



Code: Using Doc2Vec to Find Companies Aligned With Semantic Vectors

This is an ungraded practice activity.

One of the major benefits of the Doc2Vec model is the ability to find companies closest to a set of preferred terms. Rather than branching out from one company to find its closest competitors, you can combine words that describe a business and see which is closest to that theme. This customized approach to semantic vectors can provide significant advantages over more traditional methods of portfolio construction, as it allows you to craft your own theme and associations rather than relying on default associations made by a particular algorithm and/or human judgment.

Now that you have seen how the `most_similar` Doc2Vec function can be used to build lists of companies (`docvecs.most_similar`) and words (`wv.most_similar`) in alignment with a given vector input, you are ready to build a list of companies aligned to a custom set of semantic vectors based around a particular theme. To achieve that alignment, however, you must first identify and build a list of words relevant to that theme to incorporate into your semantic vectors.

In this activity, you will build two sets of custom semantic vectors to find companies in alignment with the ARKG and BOTZ thematic portfolios respectively, identifying and incorporating words most relevant to the themes of both ETFs.

Activity Instructions: Using the sandbox below, address the following tasks and questions to build custom semantic vectors with `d2v.wv` and `d2v.wv.most_similar` for the BOTZ and ARKG ETFs and run them through `d2v.docvecs.most_similar` to find the companies most closely aligned to their respective portfolio themes for the Top 1000 companies. Once you feel confident in your response, click the **Solution** button located beneath the sandbox to reveal the answers.

1. Run `d2v.docvecs.most_similar` on the semantic vector built through `d2v.wv` for `robotics` + `artificial_intelligence` to find companies aligned with the BOTZ ETF theme.



- List the top 10 companies nearest `robotics` + `artificial_intelligence`.
 - Is this list relevant given the semantic vector input?
2. Run `d2v.wv.most_similar` for `robotics` + `artificial_intelligence`.
- List the top 10 most similar words identified for `robotics`.
 - List the top 10 most similar words identified for `artificial_intelligence`.
 - How relevant are those words in the identification of comparable companies for `robotics` and `artificial_intelligence` respectively?
3. Print a series of word frequencies for words specific to the types of comparable companies desired for comparison with the BOTZ ETF theme.
4. Build custom semantic vectors for `robotics` and `artificial_intelligence` with the most relevant words identified and run `d2v.docvecs.most_similar` to find companies in alignment.
- List the top 10 companies nearest your collection of custom semantic vectors.
 - Is this list relevant given the semantic vector input?
 - How does this list compare to your original list of companies for the BOTZ ETF theme?
5. Run `d2v.docvecs.most_similar` on the semantic vector built through `d2v.wv` for `genomic` + `crispr` + `bioinformatics` and `molecular_diagnostics` to find companies aligned with the ARKG ETF theme.
- List the top 10 companies nearest those semantic vectors.
 - Is this list relevant given the semantic vector input?
6. Run `d2v.wv.most_similar` for `genomic`, `crispr`, `bioinformatics`, and `molecular_diagnostics`.
- List the top 10 most similar words identified for `genomic`.
 - List the top 10 most similar words identified for `crispr`.
 - List the top 10 most similar words identified for `bioinformatics`.
 - List the top 10 most similar words identified for `molecular_diagnostics`.
 - How relevant are those words in the identification of comparable companies for `genomic`, `crispr`, `bioinformatics`, and `molecular_diagnostics` respectively?
7. Build custom semantic vectors for `genomic`, `crispr`, `bioinformatics`, and `molecular_diagnostics` with the most relevant words identified in your `d2v.wv.most_similar` test and run `d2v.docvecs.most_similar` to find companies in alignment.
- List the top 25 companies nearest your collection of custom semantic vectors.
 - Is this list relevant given the semantic vector input?
 - How does this list compare to your original list of companies for the ARKG ETF theme?



Please access this Jupyter Notebook exercise in the course.

Solution



1. Run `d2v.docvecs.most_similar` on the semantic vector built through `d2v.wv` for `robotics` + `artificial_intelligence` to find companies aligned with the BOTZ ETF theme.

a. List the top 10 companies nearest `robotics` + `artificial_intelligence`.

Answer: National Instruments 45203010; PTC 45103010; Ansys 45103010; Cognex 45203010; Altair Engineering 45103010; Cadence Design Systems 45103010; Autodesk 45103010; Synopsys 45103010; Zebra Technologies 45203010; Keysight Technologies 45203010.

b. Is this list relevant given the semantic vector input?

Answer: Broadly, yes. Most of these companies are software providers that design hardware and software, including advanced simulations which help in the training of AI systems. Some of these companies also manufacture sensors and vision systems in support of automation.

2. Run `d2v.wv.most_similar` for `robotics` + `artificial_intelligence`.

a. List the top 10 most similar words identified for `robotic`.

Answer: `imaging` (0.930), `calibration` (0.924), `computer_aided` (0.914), `robotics` (0.905), `motion` (0.905), `visible` (0.901), `machine_vision` (0.899), `supra` (0.897), `guided` (0.894), `peripheral` (0.892).

b. List the top 10 most similar words identified for `artificial_intelligence`.

Answer: `ai` (0.976), `machine_learning` (0.973), `combine` (0.970), `transform` (0.969), `accelerates` (0.967), `agile` (0.965), `smarter` (0.965), `simplify` (0.959), `dynamic` (0.959), `uncover` (0.957).

c. How relevant are those words in the identification of comparable companies for `robotics` and `artificial_intelligence` respectively?

Answer: Very relevant. While there are a variety of derivatives (e.g., `robotic`, `robotics` or `artificial_intelligence`, `ai`, `machine_learning`, `ai_powered`), there are also words that can hone your focus on the specific type of list you want to build. For example, `robotic` has relevant words around `sensor`, `imagers`, `motion`, or `visible`, all of which are components used to gather input for robotics applications. Adding different words targeted to your goals will shape the final vector to find the most relevant comparable companies. However, there are also a number of words that do not add value, such as `simplify` or `faster`, which are best ignored when trying to construct your assembly of word vectors.



4. Build custom semantic vectors for `robotics` and `artificial_intelligence` with the most relevant words identified and run `d2v.docvecs.most_similar` to find companies in alignment.

a. List the top 10 companies nearest your collection of custom semantic vectors.

Answer: PTC 45103010; National Instruments 45203010; Palantir 45103010; Ansys 45103010; Splunk 45103010; Autodesk 45103010; Anaplan 45103010; UIPath 45103020; Elastic 45103010; New Relic 45103010.

b. Is this list relevant given the semantic vector input?

Answer: Yes, this list of companies broadly develop components and systems that are either used in robotics or provide artificial intelligence applications.

c. How does this list compare to your original list of companies for the BOTZ ETF theme?

Answer: You can view the most recent holdings from the BOTZ ETF from their website: https://www.globalxetfs.com/funds/botz/?download_full_holdings=true. There is very little overlap between the two lists. Part of that mismatch is because the BOTZ ETF invests in foreign companies as well, as there are a number of Japanese and Korean holdings. However, even for U.S. stock there is no overlap, indicating that your approach is identifying a different group of robotics companies. This is likely a result of the dataset, which is limited to the Top 1000 companies.

5. Run `d2v.docvecs.most_similar` on the semantic vector built through `d2v.wv` for `genomic` + `crispr` + `bioinformatics` and `molecular_diagnostics` to find companies aligned with the ARKG ETF theme.

a. List the top 10 companies nearest those semantic vectors.

Answer: Perkinelmer 35203010, Agilent Technologies 35203010, Azenta 45301010, Ortho Clinical 35101010, Illumina 35203010, Bruker 35203010, Danaher 35203010, Thermo Fisher Scientific 35203010, Bio-Rad Laboratories 35203010, Marava LifeS 35203010.

b. Is this list relevant given the semantic vector input?

Answer: Yes, these companies are almost all life science organizations using genetics. The only exception is Azenta, which is a software company that provides systems used by genomics companies.

6. Run `d2v.wv.most_similar` for `genomic`, `crispr`, `bioinformatics`, and `molecular_diagnostics`.



- a. List the top 10 most similar words identified for **genomic**.

Answer: **genetic** (0.988), **translational** (0.979), **microarray** (0.977), **genotyping** (0.972), **biomarkers** (0.971), **biology** (0.969), **dna** (0.968), **biomarker** (0.968), **guardant** (0.966), **gene** (0.966).

- b. List the top 10 most similar words identified for **crispr**.

Answer: **base_editing** (0.978), **crispr_casnnnn** (0.978), **gene_editing** (0.972), **product_candidates** (0.963), **allogeneic** (0.960), **ex_vivo** (0.959), **nk** (0.958), **base_editors** (0.957), **immunotherapy** (0.955), **immuno_oncology** (0.953).

- c. List the top 10 most similar words identified for **bioinformatics**.

Answer: **ng** (0.966), **researcher** (0.965), **sequencing** (0.959), **microarray** (0.956), **genomic** (0.941), **sample** (0.935), **interrogate** (0.935), **molecular** (0.932), **modality** (0.930), **gene_expression** (0.924).

- d. List the top 10 most similar words identified for **molecular_diagnostics**.

Answer: **molecular** (0.980), **cytometry** (0.978), **reagent** (0.976), **proteomics** (0.970), **microbiology** (0.967), **immunoassay** (0.965), **hcp** (0.964), **biological** (0.963), **interrogate** (0.962), **sample** (0.958).

- e. How relevant are those words in the identification of comparable companies for **genomic**, **crispr**, **bioinformatics**, and **molecular_diagnostics** respectively?

Answer: Very relevant, in addition to derivatives of the words, there are technical terms that are related to the descriptions of life science activities.

7. Build custom semantic vectors for **genomic**, **crispr**, **bioinformatics**, and **molecular_diagnostics** with the most relevant words identified in your **d2v.wv.most_similar** test and run **d2v.docvecs.most_similar** to find companies in alignment.

- a. List the top 25 companies nearest your collection of custom semantic vectors.

Answer: Marava 35203010; Illumina 35203010; OncBiomune 35201010; Ginko Bioworks 35201010; Beam Therapeutics 35201010; Intellia Therapeutics 35201010; Charles River Labs 35203010; Pfizer 35202010; Vertex Pharmaceuticals 35201010; Regeneron Pharmaceuticals 35201010; Catalent 35202010; Exact Sciences 35201010; Arrowhead Pharmaceuticals 35201010; 10x Genomics 35203010; Guardant Health 35102015; Bristol-Myers Squibb 35202010; Bio-Rad Laboratories 35203010; PureTech Health 35201010; Bio-Techne 35203010; Cerevel Therapeutics 35201010; Lilly (Eli) 35202010; Sarepta Therapeutics 35201010; Zoetis 35201010; Abbot Laboratories 35101010; Quidel 35101020.



b. Is this list relevant given the semantic vector input?

Answer: Yes, this list is significantly aligned with the genomics field.

c. How does this list compare to your original list of companies for the ARKG ETF theme?

Answer: You can view the **most recent holdings for the ARKG ETF** from their website. There is an overlap of nine names from the top 25 provided by the algorithm, which constitutes 28.4% of the fund.

[Back to Table of Contents](#)



Discuss: Thematic Portfolio Construction

Discussion topic

Thematic ETFs are in some ways a perfect demonstration of the power of algorithms like Doc2Vec and LDA to replicate or even construct a diverse list of companies across industry sectors based on their relevance to a desired theme. While this process can be iterative in nature, the ability to refine inputs and customize outputs specific to your needs provides a significant advantage over more subjective methods for construction.

In the case of Doc2Vec, semantic vectors can be developed and refined specific to keywords of your choice, providing company associations that may not have been readily apparent through traditional means. However, your first attempt at replicating the ARKG ETF focus yielded an overlap of just 28.4%, which shows that your semantic vectors captured some, but not all, of the companies present in that fund.

While a portion of that difference was a result of the limited dataset employed (Top 1000 domestic companies), there were still a number of companies missed that might have been incorporated had you further refined semantic vectors. Take a moment and share your thoughts on the topic with your classmates, structuring your response around the following question:

How would you adjust your approach to semantic vectors to create a list of stocks more reflective of the ARKG ETF theme?

- What words would you add/remove from your list of semantic vectors?
- How deep of a list of companies would you provide for comparison? (*You evaluated the top 25 companies, whereas the ARKG ETF has 43*)
- How would you weight your list of semantic vectors? (*The ARKG ETF has significant weight in the Exact Sciences, favoring companies like Beam*)

Instructions



You are required to participate in all course discussions. To be able to see and respond to the posts of your peers you must post a response. Keep your initial posts focused and brief; no more than 200 words. Actively participate by asking questions of your peers and responding to their posts with your own comments. *To receive full credit you are only required to post an initial response to the discussion prompt, although it is highly recommended that you also reply to the posts of at least two of your peers.*

To participate in this discussion: Click Reply to post a comment or reply to another comment.

Your initial post will be assessed according to the following criteria:

- Comprehension - *Demonstrates an understanding of course content.*
- Synthesis - *Demonstrates ability to synthesize course content within the discussion topic.*
- Originality - *Utilizes original thinking.*
- Development - *Clearly articulates and fully addresses all requirements.*

Please consider that this is a professional forum; courtesy and professional language and tone are expected. When you submit work for academic credit, you are certifying that the work is your own. All outside assistance should be acknowledged, and your original academic work and ideas truthfully reported at all times. Whether intentional or unintentional, plagiarism is a serious violation of **Johnson's Honor Code** and **Cornell's Code of Academic Integrity**. Avoid plagiarism by doing original work, and always citing, acknowledging, and giving credit to all sourced materials.

[**Back to Table of Contents**](#)



Assignment: Module 4.2 Assignment: Using Doc2Vec Loadings to Find Comparable Companies

As you may remember, statement analysis often involves a comparison of publicly traded stocks against peers to determine the attractiveness of an investment. However, when building a list of comparable companies, it is best to use and combine words that describe a particular concept/business, rather than starting with one company and branching out to find those closest.

Now that you have built lists of comparable companies and words for a given company, word, or set of word inputs, and combined those skills to customize semantic vector input and find companies specific to a given theme, how might you adapt your approach for a different theme or set of business descriptions?

In this assignment, you will attempt to replicate the holdings of two thematic ETFs with the Middle 1000 companies, identifying and incorporating words relevant to those themes into your own customized semantic vectors to build a list of comparable companies and assess their alignment to both investment portfolios.

Completion of this assignment is a course requirement.

Instructions:

1. Download the **Module 4.2 Assignment** template to use with the sandbox below.
2. Run `d2v.docvecs.most_similar` to build a list of comparable companies, run `d2v.wv.most_similar` to build a list of comparable words, and apply both skills to build custom semantic vectors for the SOCL and ARKG ETF themes to find the companies most closely aligned to their respective portfolio themes for the Middle 1000 companies.
3. Use the output to address tasks in the assignment template.
4. Save your work locally as an accepted file type (.doc, .docx, .txt, .pdf) with the following naming convention:
BANA5210_LastName,FirstName_Module4.2Assignment. No other file types will be accepted for submission.



5. Once you are confident in your responses, click the **Start Assignment** button to upload and submit that file for manual review.

Before you begin:

Please consider that this is a professional forum; courtesy and professional language and tone are expected. When you submit work for academic credit, you are certifying that the work is your own. All outside assistance should be acknowledged, and your original academic work and ideas truthfully reported at all times. Whether intentional or unintentional, plagiarism is a serious violation of **Johnson's Honor Code** and **Cornell's Code of Academic Integrity**. Avoid plagiarism by doing original work, and always citing, acknowledging, and giving credit to all sourced materials.

Please access this Jupyter Notebook exercise in the course.

[Back to Table of Contents](#)



Module Wrap-up: Use Doc2Vec Loadings to Find Comparable Companies

In this module, you built lists of comparable companies by similarity in semantic meaning, evaluating the relevance of activity associations made against GICS subindustry groupings. You then built lists of words comparable to single and combined vectors to explore the mechanism underlying Doc2Vec semantic associations, customizing your own semantic vectors to build lists of companies relevant to a particular theme. Finally, in the module assignment, you identified words in the business description corpus of the Middle 1000 companies reflective of the SOCL and ARKG thematic investment portfolios, incorporated them into custom semantic vectors, and built lists of companies comparable to those themes specifically.

[Back to Table of Contents](#)



Module Introduction: **Apply the Doc2Vec Algorithm**



While the Doc2Vec model may have valuable advantages for the construction of investment portfolios based on a specific theme, there is still some benefit to the topic grouping provided by the LDA algorithm. Given that GICS is the current benchmark for the classification of companies by activity, it is important that you have a mechanism by which to evaluate alignment of your Doc2Vec model with those groupings specifically.

In this module, you will explore ensembling or "stacking" as a way to incorporate additional algorithms into your Doc2Vec model, allowing for companies to be grouped by topic and evaluated against GICS classifications. You will first incorporate the **KMeans** algorithm into your model, clustering companies by topic to evaluate their alignment with GICS. You will then incorporate PCA as a means to enhance the relevance of those clusters, comparing results against your first clustering attempt. Finally, in the module assignment you will perform those same operations on your Doc2Vec model for the Middle 1000 companies, evaluating for relevance against the LDA model with respect to GICS alignment.

[**Back to Table of Contents**](#)



Watch: Introduction to Clustering Algorithms

While the Doc2Vec algorithm does not generate topics in the same way that the LDA algorithm did, there is a utility to that approach that bears consideration. As you may recall, LDA topic groupings were compared within GICS industry groups and vice versa to get a sense of their similarities, differences, and overall relevance to activity alignment. In this video, Mr. Meredith introduces a new algorithm to set vectors up for that same task, clustering numerical data into topics for comparison against GICS.

Video Transcript

One of the exercises that we did in the previous set of lectures was the idea of creating groupings or topics that came from the underlying data and having the machine learning algorithms determine how to group the companies together. We used LDA, and if you remember LDA had used the words and the word frequency to mathematically separate the best description of the topics from its separation with the best description of the words within those topics, the maximum probability of that.

It's not the only way to go about and create topics. What I want to introduce here are clustering algorithms that do the similar exercise on numerical data. The idea is you can use your Doc2Vec algorithm and you can attempt to have it where you're taking the numerical loadings that are coming from the underlying text and semantic vectors and have that where you're trying to then cluster companies and create groupings inside of them.

There's multiple clustering algorithms out of there. What we're going to start with is one that's an extremely popular one called the **KMeans** algorithm. The **KMeans** algorithm — again, you can learn a ton about it by going out in Wiki and reading about it. But I think the key part is that you're going to feed in the information, the numerical loadings, from doc2vec, and it'll mathematically separate them out. Out of those 150 loadings, try to create clusters that have the most distance between them.

In this case, what we're going to start with is if we go through and take the KMDF, we're going to create that d2v copy of it. The main reason for that copy is iteratively if



you make a mistake, you can start over from the same point. We're going to import inside of this, the `sklearn.cluster import KMeans`, the `sklearn.preprocessing import StandardScaler`. I'll explain why we want that in a second.

What we're going to do then is from our `doc2vec` algorithm that we had before, we're going to then take what the index to identity is. This is simply a matter of saying, give me what the identifier is inside of `doc2vec` — in this case, it's actually the clean description that we're using for that — then have it where we're going to append the `doc2vec` vectors from that. This is one where we're making sure that our `x` and `y` are coming through and saying, let's make sure that we're aligning those closely and making sure we're not creating a problem with ourselves just by how we're extracting the data.

This standardized scalar I mentioned is when we're going to standardize all the loadings to a normal 01. The reason for this is that the `KMeans` algorithm is highly sensitive to outliers. It's one where if there is some one data point on the outside or on either direction, it can wind up having it where it will be influenced, the broader average, just by having a few data points that are off.

With that scalar, what we're going to do is push that into the K-means algorithm. Again, incredibly powerful having the open-source software. We know that we're taking in here the `sklearn.cluster KMeans` algorithm. We're going to first of all create it. We're going to predefine 24 clusters. We're going to give it a number of iterations that we're going to walk through and hyperparameter tune. You can play with this, send a few number of iterations, large number of iterations. The number of clusters you can play with, a lot of them or a few of them. In this case, we're trying to replicate what we did with the 24 industry groups.

We're going to `fit` and have the `scaled_X`. We're going to get our predictions. One thing to note about this model is it's not a partial inclusion. You're going to get one assignment to one cluster, and that will be applied to it. We're going to then create another dataframe, which is going to be giving my descriptions and my predictions, that coming from the `d2v.docvecs.index2entity` up here, the prediction. We're going to merge that back in.

Then we're going to see. We're going to wind up here having this group of 24 different industry groups that are basically `d2vIndGrp` industry groups. You can see the ranges of



these are somewhat similar, small to large. It's not an equal distribution of them. Not as small as what we had with our food group, but some that are down as low as 46 or 43, our highest being 728 and 463. What we want to do in the next video is start evaluating and see how good of a job the `KMeans` algorithm did on creating topics or clusters out of the `doc2vec` data that we provided to it.

[Back to Table of Contents](#)



Watch: Evaluating Clustering Algorithms

With your vectors clustered into topics by the `KMeans` algorithm, you can now compare your results against GICS in a manner similar to that of LDA topic groupings. In this video, Mr. Meredith demonstrates how `KMeans` clusters and GICS industry groupings can be expressed within one another, allowing for holistic evaluation of company alignment by business activity.

Note: For the purpose of readability, words from the Jupyter Notebook in this video transcript are not stylized.

Video Transcript

One of the benefits of the LDA algorithm was this interpretability, the clean alignment of the word frequencies with how the topics were formed. What we're going to do in evaluating this is showing the groupings that are coming through. But we need to approximate essentially how close we're getting with the word frequencies and put those inside of the groups. We also need to figure out how to show the companies that are inside of them and figure out the alignment.

I will say it went through a couple of different iterations on this, getting the interpretability and aligning it was not as straightforward. Wound up saying, what are the closest companies to the vectors and the groupings that come through? In `KMeans`, you get a center group that is the 150 loadings and what the middle is, and then you can get the distance to it. But you wind up with a lot of small and micro cap names that wasn't necessarily as intuitive for what would be as a user of this, so I sorted the ones that are in the group by market cap. It may not be perfect, but it's something we can do to try to figure out how well aligned these are.

When we start walking in, let's look through the code, first of all. From this import collections counter, we're going to use that just to get a count of the words that are inside of this. What we're going to take a look at as we also have our word frequency, we're going to take a look at topic names ranged from 0 to 24. For every one of these, we're going to print out which cluster it is. We're then going to apply and get the word frequency for each one of those. We're going to create a `smallD2` for our small data



frame, where it's only going to be in that group. We're going to apply the word frequency. We then take the largest in that by market cap. We're going to supply the top five to see how close we are. We're going to take the word frequency inside of that `smallD`, and then basically put through and show that appended to an array that's the number of words. By the time we're done, we're going to get a dataframe. It's going to have the industry group, the top five companies by market cap, and the top five words attached to that.

For cluster 1, what we see is Berkshire Hathaway, Progressive, MetLife, Prudential, and Aflac. This looks pretty good. Insurance, coverage, policy, investment, reinsurance — that's a pretty tight list there of what we have for our insurance grouping.

For our group 1, JP Morgan, Bank of America, Morgan Stanley, Citigroup, PNC Financial Services, and loan, bank, banking, investment, deposit. Very clean. Microsoft, Adobe, Cisco for cloud, software enterprise. Ecolab, Dow, DuPont for chemical, paper, packaging. We're getting our materials grouping that's coming through. Disney, Comcast, AT&T for content, programming, video. We're pretty clean on this. Eli Lilly, Merck, Amgen for treatment, patient, patents, or health care. Tesla, Home Depot, Honeywell for vehicle, power, automotive. Coca-Cola, Pepsi, Philip Morris for food, store, ingredient. UPS, Raytheon for vehicle, aircraft.

Here's where we encounter our first problem. For this `KMeans` cluster number 9, we see Apple, Amazon, Procter and Gamble, Netflix, Costco. It's got the words of loan, banking, investment, bank, insurance. This is the one group that pulls out. If you remember, when we were walking through the counts, that group 9 is the largest at 728 companies. What's happening is this is our one group that's coming through where there's not necessarily a clean clustering inside of it. That cluster 9 is one where I really had a hard time figuring it out.

By the way, the loan, banking, investment, bank, the reason that those are the top names, if you remember from the previous in the word frequency, the number one word overall is loan. These are the most common in what is like a random grouping of companies that's coming through in our clustering.

Now part of this is the algorithm itself is not getting a clean read on it. When we go through on our GICS industry, what we can see is that there's just a percentage of companies that's coming through in this industry group 9 with loan, banking,



investment, where it's going to have some random distribution. For energy, for the most part, outside of that topic 9, you're getting your natural gas, power, pipeline, natural gas.

This is a piece of code where we're going to walk each of the GICS industry groups. We're going to print out what the topics are that are aligned inside of that and walk through those topics to see how well they're aligned with GICS. Number 9, in there is number 2, otherwise it's cleanly aligned. For this case, for our Materials, chemical, paper, natural gas, powerful for materials outside of that group.

For Capital Goods, which you remember is a diversified group, we're actually getting some cleanness on vehicle power, automotive energy up at the top. But again, still the number 9 coming through, commercial professional services as our number 1. Other ones are cleaner.

Transportation, pretty good outside of the industry group 9. We have the word vehicle, power, automotive, and automobile components. Consumer Durables, store, retailer, accessory, wholesale. Consumer Services winding up at the top here with this number 9 in our group, but you're getting your casino, hotel, etc.

I guess it's a long-winded way of saying we do have this one thread in here of this group that doesn't seem to be getting us a clean. But the rest of the topic — by the way, the number of companies we're talking 700 plus. About 20% or less on average of the companies are having a hard time getting a clean classification. But **KMeans** is classifying the other 80% pretty cleanly. When we walk through and evaluate we think, not bad. But we have some work to do on improving that algorithm.

[Back to Table of Contents](#)



Code: Evaluating Clustering Algorithms

This is an ungraded practice activity.

LDA assigns output to groupings, which can have benefits in performance analysis as well as trying to control for common risks in portfolio construction. While the Doc2Vec numerical landscape does not include groupings as part of the output, you can utilize the `d2v` loadings to generate a set of clusters with similar utility as LDA topic groupings.

To accomplish that, the output of your `d2v` model has to be fed into a clustering algorithm. This act of taking one machine learning model and utilizing it within another is a common example of ensembling or "stacking." In this activity, you will utilize the `KMeans` clustering algorithm to cluster your vectors into topics for comparison against GICS, evaluating their relative alignment with respect to business activity.

Activity Instructions: Using the sandbox below, address the following tasks and questions to run the `KMeans` clustering algorithm on `d2v` loadings to classify the Top 1000 companies by topic probability, evaluate the distribution of `KMeans` topic clusters within GICS industry groups and GICS industry groups within `KMeans` topic clusters, and compare the relative efficacy of groupings against those of the LDA model. Once you feel confident in your response, click the **Solution** button located beneath the sandbox to reveal the answers.

1. Run a baseline `KMeans` cluster.
 - List the company counts by cluster.
 - Which is the largest?
 - Which is the smallest?
2. Print out the top 10 companies by market cap and the top 5 word counts within each cluster.
 - List the top 5 companies in the largest cluster.
 - Does this group seem coherent?
 - List the top 5 companies in the smallest cluster.
 - Does this group seem coherent?



3. Print out the cluster assignments, organized by GICS industry group, and accounting for the number of companies and percent relation to GICS by cluster.

- Identify which GICS industry group is the most consistent by **KMeans** cluster.
- Identify which GICS industry group is the least consistent by **KMeans** cluster.
- How consistent are GICS industry groups overall with respect to **KMeans** cluster grouping/percent?

4. Print out the GICS industry groups, organized by **KMeans** cluster, and accounting for the number of companies and percent relation to **KMeans** cluster by GICS industry group.

- Identify which **KMeans** cluster is the most consistent by GICS industry group.
- Identify which **KMeans** cluster is the least consistent by GICS industry group.
- How consistent are **KMeans** clusters overall with respect to GICS industry grouping/percent?

5. How would you characterize the efficacy of applying **KMeans** + **d2v** against GICS given the results of your comparisons thus far?

6. How would you characterize the efficacy of **KMeans** + **d2v** against the LDA model for the Top 1000 companies in comparisons made against GICS?

Note: Topic groupings will be different for each run of the model; hence, your results may not reflect the activity solution.

Please access this Jupyter Notebook exercise in the course.

Solution



1. Run a baseline **KMeans** cluster.

a. List the company counts by cluster.

Answer: 0(45), 1(28), 2(37), 3(79), 4(33), 5(35), 6(25), 7(46), 8(168), 9(12), 10(7), 11(37), 12(4), 13(84), 14(5), 15(15), 16(17), 17(32), 18(85), 19(26), 20(11), 21(65), 22(22), 23(37).

b. Which is the largest?

Answer: Group 8 with 168 companies.

c. Which is the smallest?

Answer: Group 12 with 4 companies.

2. Print out the top 10 companies by market cap and the top 5 word counts within each cluster.

a. List the top 5 companies in the largest cluster.

Answer: Cluster 8 - Home Depot, Proctor & Gamble, Costco, McDonald's, Wells Fargo.

b. Does this group seem coherent?

Answer: No, these companies are not related to one another.

c. List the top 5 companies in the smallest cluster.

Answer: Cluster 12 - Freeport McMoran, Newmont, Southern Copper, Mosaic.

d. Does this group seem coherent?

Answer: Yes, these are very specific mining companies.

3. Print out the cluster assignments, organized by GICS industry group, and accounting for the number of companies and percent relation to GICS by cluster.

a. Identify which GICS industry group is the most consistent by **KMeans** cluster.

Answer: Industry Group 4010 - Banks has 95.8% alignment with cluster 3.

b. Identify which GICS industry group is the least consistent by **KMeans** cluster.

Answer: GICS Industry Group 2020 seems the least consistent, as it is spread across eight different clusters with the top cluster accounting for just 38.7% of the companies in the group.

c. How consistent are GICS industry groups overall with respect to **KMeans** cluster grouping/percent?

Answer: There is some alignment between GICS and **KMeans**, but it is not as clean as the LDA exercise. The most prevalent topics in sectors are sometimes



aligned, for example the top group for Energy is `pipeline, natural_gas, crude_oil`, and the top group for Materials is `industrial, material`. The issue is that topic 8 is also the most prevalent for a number of GICS Industry groups (e.g., Transportation, Retailing, Food & Staples Retailing, Real Estate).

4. Print out the GICS industry groups, organized by `KMeans` cluster, and accounting for the number of companies and percent relation to `KMeans` cluster by GICS industry group.

a. Identify which `KMeans` cluster is the most consistent by GICS industry group.

Answer: Clusters 4, 6, 9, and 12 were in 100% alignment with GICS industry groups for Pharmaceuticals, Biotechnology & Life Sciences; Energy; Media & Entertainment; and Materials.

b. Identify which `KMeans` cluster is the least consistent by GICS industry group.

Answer: Cluster 8 was the least aligned, as it was spread across 19 of the 24 GICS industry groups with the top industry group accounting for just 12.5% of the companies in the cluster.

c. How consistent are `KMeans` clusters overall with respect to GICS industry grouping/percent?

Answer: While there are a number of clusters that are aligned with GICS classifications, there are a couple that are not well formulated.

5. How would you characterize the efficacy of applying `KMeans` + `d2v` against GICS given the results of your comparisons thus far?

Answer: The best description would be that this was a crude first attempt. While the algorithm is making some associations, there is a significant amount of noise in the clustering exercise.

6. How would you characterize the efficacy of `KMeans` + `d2v` against the LDA model for the Top 1000 companies in comparisons made against GICS?

Answer: The LDA algorithm was far more effective clustering with minimal inputs than the Doc2Vec algorithm was.

[Back to Table of Contents](#)



Watch: Putting it All Together

As you near the end of this course experience, take a moment to reflect on the key concepts and methods you've practiced along the way. In this video, Mr. Meredith compares Doc2Vec and LDA, contextualizing their relevance to topic modeling and discussing potential improvements to model performance. While you may have a preference for one method over the other, their respective utility is dependent on your ability to get the output you need.

Video Transcript

We're nearing the end of our time on topic modeling. I just want to review some of the things that we went over in this series of lectures. First, talking to the benefits of Doc2Vec, where for competitive names and having it where you're trying to identify companies that are the closest to it, in the LDA algorithm we went through where first we built the topics and then did a form of distance to those topics, whereas in the Doc2Vec we are starting from the semantic vectors and working towards the similarity matrix. We can do that from, first of all, the 150 loadings of those semantic meanings. We can also then build it from the actual words themselves with the semantic vec words. There's some power in that framework of Doc2Vec to try to identify companies that are the closest to any sort of topic or theme or company that you are trying to identify.

When we are talking through though the benefits of clustering and trying to figure out groupings, which again, those ideas of competitive names is used more in pairs trading or financial statement analysis. Clustering is used more in risk controls and identifying portfolio construction, areas that you want to maybe shape weights around.

This is one where LDA seems to have some benefits over Doc2Vec, that idea that the LDA is using what the word frequencies and topic frequencies are and separating those out, having it where there is distance between them, and there's a real cleanliness to those groupings that was coming through versus what we were trying to do with a Doc2Vec loadings and having it where we were creating clusters off of them.



It doesn't mean that the K-means and doc2vec has failed by any stretch of the imagination. There's some work to do to get this where it may be improved and get better. There's things that we can work on, and hopefully the framework we gave you on machine learning projects gives you ideas on how to go through and improve that, things like feature engineering, cleaning up the inputs, maybe having it where the Doc2Vec loadings, fewer or more, may help what's going on inside of it.

The algorithms selection where maybe K-means isn't the right clustering algorithm. Maybe we want to use some other ones out there, ones that maybe have more partial inclusion, like a fuzzy K-means algorithm or other ones that might define the number of topics for you. DBSCAN is a good example of that. The idea is the algorithms selection might be the place to put it.

The hyperparameter tuning, maybe tweaking the K-means, having it where you're fitting it less or more, working through the number of topics inside of that, they may also help on this. But those are the things, the tools and the levers, that we want you thinking about when you're going through and trying to take a project from the beginning step and bringing it all the way to completion.

By the way, one other potential area of improvement that we haven't talked through is ensembling. There may be ways that you can combine the strengths of LDA with the strengths of Doc2Vec and figure out how to put those together to get you a better outcome for whatever you're trying to predict. With that, we hope we've set you up well for future endeavors using language and topic modeling and the skills we've provided.

[**Back to Table of Contents**](#)



Code: Ensembling for Improved Clustering

This is an ungraded practice activity.

Building a clustering algorithm is a whole new machine learning project in itself, and while the first results were mixed, that does not mean that you should not attempt to improve the results. Reflecting back to the beginning of the course, there are six steps to any machine learning project. What are some things you could do that might improve upon your initial clustering?

- 1. Data Preparation:** Do you need to gather more descriptive information on the Top 1000 companies?
- 2. Label Selection:** Are you choosing the right thing to try and predict the grouping of companies?
- 3. Feature Selection:** Are `d2v` loadings the right input for the algorithm? Is there a way to modify features such that application of the `KMeans` algorithm is more appropriate?
- 4. Algorithm Selection:** Is the `KMeans` cluster the best algorithm to try and determine groupings?
- 5. Hyperparameter Tuning:** Are there parameters within the `KMeans` clustering algorithm that can make the model more effective in its groupings?
- 6. Ensembling:** Would combining multiple clustering algorithms help for a more effective model?

In this activity, you will address the "Feature Engineering" aspect of Feature Selection, which is the act of manipulating or transforming the data input to make it more "readable." `KMeans` is sensitive to outliers, so simplifying it through dimension reduction may help to improve outcomes.

Activity Instructions: Using the sandbox below, address the following tasks/questions to run the `KMeans` clustering algorithm on `d2v` loadings that have been reduced from 150 dimensions to 3 through PCA to classify the Top 1000 companies by topic probability. Then evaluate the distribution of `KMeans` topic clusters within GICS industry groups and GICS industry groups within `KMeans` topic clusters, and compare the relative efficacy of groupings against those of both the LDA model and your first



attempt at **KMeans** clustering. Once you feel confident in your response, click the **Solution** button located beneath the sandbox to reveal the answers.

1. Run a baseline **KMeans** cluster on PCA reduced **d2v** loadings.
 - List the company counts by cluster.
 - Which is the largest?
 - Which is the smallest?
2. Print out the top 10 companies by market cap and the top 5 word counts within each cluster.
 - List the top 5 companies in the largest cluster.
 - Does this group seem coherent?
 - List the top 5 companies in the smallest cluster.
 - Does this group seem coherent?
3. Print out the cluster assignments, organized by GICS industry group, and accounting for the number of companies and percent relation to GICS by cluster.
 - Identify which GICS industry group is the most consistent by **KMeans** cluster.
 - Identify which GICS industry group is the least consistent by **KMeans** cluster.
4. Print out the GICS industry groups, organized by **KMeans** cluster, and accounting for the number of companies and percent relation to **KMeans** cluster by GICS industry group.
 - Identify which **KMeans** cluster is the least consistent by GICS industry group.
5. How would you characterize the efficacy of applying **KMeans** + **PCA** + **d2v** against GICS given the results of your comparisons thus far? Does this seem better than your first attempt?

Note: Topic groupings will be different for each run of the model; hence, your results may not reflect the activity solution.

Please access this Jupyter Notebook exercise in the course.

Solution



1. Run a baseline **KMeans** cluster on PCA reduced **d2v** loadings.

a. List the company counts by cluster.

Answer: 0(32), 1(28), 2(20), 3(1), 4(41), 5(288), 6(3), 7(58), 8(98), 9(14), 10(25), 11(21), 12(20), 13(31), 14(33), 15(10), 16(80), 17(25), 18(9), 19(13), 20(30), 21(22), 22(21), 23(77).

b. Which is the largest?

Answer: Group 5 with 288 companies.

c. Which is the smallest?

Answer: Group 3 only has a single company.

2. Print out the top 10 companies by market cap and the top 5 word counts within each cluster.

a. List the top 5 companies in the largest cluster.

Answer: Cluster 288 - Apple, Alphabet, Amazon.com, Meta Platforms, Johnson & Johnson, Home Depot, Walmart, Procter & Gamble, Pfizer, Netflix.

b. Does this group seem coherent?

Answer: No, the same problems from the original clustering exercise are still clearly apparent.

c. List the top 5 companies in the smallest cluster.

Answer: Cluster 12 - Freeport McMoran, Newmont, Southern Copper, Mosaic.

d. Does this group seem coherent?

Answer: Yes, these are very specific mining companies.

3. Print out the cluster assignments, organized by GICS industry group, and accounting for the number of companies and percent relation to GICS by cluster.

a. Identify which GICS industry group is the most consistent by **KMeans** cluster.

Answer: Industry Group 4010 - Banks has 91.7% alignment with cluster 1, similar to the first exercise.

b. Identify which GICS industry group is the least consistent by **KMeans** cluster.

Answer: GICS Industry Group 4510 seems the least consistent, as it is spread across nine different clusters with the top cluster accounting for just 46.7% of the companies in the group.

4. Print out the GICS industry groups, organized by **KMeans** cluster, and accounting for the number of companies and percent relation to **KMeans**



cluster by GICS industry group.

- a. Identify which **KMeans** cluster is the least consistent by GICS industry group.

Answer: Cluster 5 was the least aligned, as it was spread across all 24 of the GICS industry groups with the top industry group accounting for just 17% of the companies in the cluster.

5. How would you characterize the efficacy of applying **KMeans** + **PCA** + **d2v** against GICS given the results of your comparisons thus far? Does this seem better than your first attempt?

Answer: Dimension reduction via PCA has a significant negative impact on **KMeans** clusters. The issue with a single cluster of unformed companies got worse, and there are no clusters with a single constituent. While this attempt at feature engineering was a failure, sometimes setbacks can give an indication of what to try next. If having less information is making the process less effective, perhaps the clustering algorithm needs more information.

[Back to Table of Contents](#)



Assignment: Module 4.3 Assignment: Applying the Doc2Vec Algorithm

As you have seen, the Doc2Vec numerical landscape does not include groupings as output, which is an important factor for both performance analysis and risk management when constructing a portfolio of comparable companies. To generate groupings similar to those output by the LDA model, **d2v** loadings must be grouped to provide similar utility for analysis.

To accomplish that, **d2v** loadings must be fed into a clustering algorithm, which requires that you incorporate an additional machine learning algorithm into your Doc2Vec model. However, your ensembling efforts do not necessarily have to be limited to any one algorithm, as multiple algorithms can often be useful to fill in gaps that others may not adequately address. By those efforts, you can effectively position your "topic" clusters such that they are aligned with GICS and relevant for the construction and management of investment portfolios.

In this assignment, you will first apply the **KMeans** algorithm to group **d2v** loadings for the Middle 1000 companies into clusters for comparison against GICS, evaluating their relative alignment with respect to activity assignment. You will then reduce the dimensionality of **d2v** loadings with the PCA algorithm, generating **KMeans** clusters on the reduced loadings to evaluate whether there is any improvement in alignment with GICS over your first attempt.

Completion of this assignment is a course requirement.

Instructions:

1. Download the **Module 4.3 Assignment** template to use with the sandbox below.
2. Run the **KMeans** clustering algorithm on **d2v** loadings to classify the Middle 1000 companies by topic probability, evaluate the distribution of **KMeans** topic clusters within GICS industry groups and GICS industry groups within **KMeans** topic clusters, and compare the relative efficacy of groupings against those of the LDA model. Then incorporate PCA into your **KMeans** clustering to perform the same steps and compare results against both LDA and your first **KMeans** effort.



3. Use the resulting output to address tasks in the assignment template.
4. Save your work locally as an accepted file type (.doc, .docx, .txt, .pdf) with the following naming convention:
BANA5210_LastName,FirstName_Module4.3Assignment. No other file types will be accepted for submission.
5. Once you are confident in your responses, click the **Start Assignment** button to upload and submit that file for manual review.

Before you begin:

Please consider that this is a professional forum; courtesy and professional language and tone are expected. When you submit work for academic credit, you are certifying that the work is your own. All outside assistance should be acknowledged, and your original academic work and ideas truthfully reported at all times. Whether intentional or unintentional, plagiarism is a serious violation of **Johnson's Honor Code** and **Cornell's Code of Academic Integrity**. Avoid plagiarism by doing original work, and always citing, acknowledging, and giving credit to all sourced materials.

Please access this Jupyter Notebook exercise in the course.

[Back to Table of Contents](#)



Module Wrap-up: **Apply the Doc2Vec Algorithm**

In this module, you explored ensembling as a way to incorporate clustering and dimension reduction algorithms into your Doc2Vec model, allowing for companies to be grouped by topic and evaluated against GICS classifications. You first incorporated the **KMeans** algorithm to cluster companies by topic for comparison with GICS, evaluating the relevance of activity associations for both approaches. You then incorporated PCA as a means to enhance the relevance of those clusters, comparing results against your first clustering attempt. Finally, in the module assignment, you performed those same operations on your Doc2Vec model for the Middle 1000 companies, evaluating for relevance against the LDA model with respect to GICS alignment.

[Back to Table of Contents](#)



Thank You and Farewell

Congratulations on completing the "NLP for Business" course!

Throughout this course, you have built practical skills to leverage natural language in business analytics. Starting with foundational aspects, such as using Pandas for data manipulation, you applied RegEx for text pre-processing, and learned how to use open-source software to build and improve model solutions for business problems.

Most of all, what I hope you took away from this course is the art of constructing a complete text modeling project, where you establish what you want to solve and work systematically through machine learning steps to build a relevant solution. This framework will aid you in any NLP endeavor that you pursue in the future.

From all of us at Cornell University, thank you for participating in this course. We hope you enjoyed the experience and wish you the best.

Sincerely,

Chris Meredith



Instructor Chris Meredith

Senior Visiting Lecturer

SC Johnson Graduate School of Management
Cornell University

